Check for updates

Original research article

Online intrusion detection in computer networks using edge-aware attentive graph neural network

Intelligent Data Analysis
1–26
© The Author(s) 2026

Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/1088467X261488186
journals.sagepub.com/home/ida



Áron Kiss[1], **Károly Nehéz**[1] **and Olivér Hornyák**[1]

Abstract

Graph Neural Network (GNN)-based intrusion detection systems (IDS) have emerged as powerful tools for modeling the structural patterns of network traffic. However, most existing methods rely on large, temporally aggregated graphs and random train-test splits, which risk information leakage from future traffic and overstate real-world performance. In practice, IDSs must operate continuously on partial, time-limited data. Bridging this methodological gap is essential for translating GNN-IDS research into deployable, real-time defense systems. This study presents an intrusion detection framework based on a GNN that represents packet-flow data as communication graphs constructed over temporally segmented time windows. The method performs periodic classifications to detect deviations from normal operation and raise early alerts on emerging attacks. Using a time-window-driven graph construction method, it effectively models the temporal evolution of communication patterns while maintaining low detection latency. For graph representation learning, an EdgeGAT-based architecture exploits both structural and edge-level features, achieving up to a 9.7% F1-score improvement over the E-GraphSAGE baseline across diverse datasets. An attention-based explainability mechanism further generates compact explanatory subgraphs that highlight the most relevant communication flows, supporting operators in understanding and mitigating attacks. The results demonstrate that the proposed approach effectively balances accuracy, responsiveness, and interpretability, offering a practical foundation for online, explainable intrusion detection in operational environments.

Keywords

Intrusion detection, computer network, graph neural network, GNN, edge-aware attention, network traffic graph, anomaly detection, explainability, cybersecurity

Received: 13 January 2026; accepted: 28 August 2026

1 Introduction

The rapid growth of digital infrastructure and increasing interconnection of systems have significantly expanded the attack surface of computer networks.[1] Cyberattacks have evolved in both scale and complexity, often bypassing traditional security measures.[2] In this context, intrusion detection systems (IDS) have become a key part of cybersecurity, complementing preventive tools such as firewalls and access control mechanisms. Unlike purely preventive solutions, IDS are designed to detect and alert on malicious activities that overcome initial defensive measures.[3] They play an essential role in identifying security threats, enabling timely response, and supporting forensic investigation. Continuous adaptation to new attack patterns and changing network environments is therefore crucial for maintaining resilient and secure IT systems.

Effective intrusion detection requires analysis of network traffic as it is generated. Modern cyberattacks can escalate quickly, and delayed detection greatly reduces the chances of limiting their impact. Online IDS processing keeps detection

[1]Institute of Information Science, University of Miskolc, Miskolc, Hungary

Corresponding author:
Áron Kiss, Institute of Information Science, University of Miskolc, Miskolc, Hungary.
Email: aron.kiss@uni-miskolc.hu

latency low, allowing security teams to act before threats spread, while offline analysis is mainly useful for forensic work and model training.[4]

Network traffic can be naturally modelled as a graph, where nodes represent host machines or sockets and edges represent observed communications.[5] This structure preserves the connections and properties of communication, enabling the detection of complex attack patterns.[6] Based on this, recent research has explored Graph Neural Network-based IDS, which can outperform traditional methods by learning topological patterns of the communication graph, that feature-based approaches often miss.[7–13]

We define online intrusion detection as the continuous analysis of network traffic under temporal constraints, where predictions must be produced from incrementally observed data without access to future traffic. In contrast to offline IDS approaches that operate on large pre-collected communication graphs, online IDS systems are expected to perform detection during system operation using only information available within a limited and recent time window. However, many current approaches treat intrusion detection as an edge-prediction task on large, aggregated network graphs spanning days or even weeks. While this may achieve high accuracy in academic evaluation papers, it is impractical for real-world use because long observation periods lead to excessive detection delays.[14]

In this study, we present a GNN-based intrusion detection method designed for online operation, that can detect anomalies in system behaviour based on temporally changing network traffic. Our model performs periodic classifications of the system's state and issues alerts whenever it detects deviations from normal behaviour. This enables continuous monitoring while leveraging graph-based learning to capture complex communication patterns. In addition, for each detected attack, the model produces an explanation graph to help human operators understand the cause of the incident and take effective countermeasures. This explainability provides insight into the model's internal decision-making process and supports informed decision-making during incident response.

The main contributions of this study are as follows:

- We propose a GNN-based IDS framework designed for realistic online operation, representing temporally bounded network traffic as directed multigraphs and performing periodic classification of the monitored system's state.
- We adapt an edge-aware attentive GNN architecture, enabling the model to jointly exploit communication structure and flow-level edge attributes.
- We introduce an attention-guided explanation method that extracts subgraphs for IDS predictions, helping operators identify the most relevant communication patterns behind alerts.
- We analyze the impact of time-window size on both detection performance and processing latency, highlighting a practical accuracy-responsiveness trade-off in online IDS deployment.
- We show that the proposed method outperforms E-GraphSAGE baseline in both binary and multiclass classification, achieving relative F1-score improvements of up to 9.74% and 6.79%, respectively, across two large-scale network intrusion datasets.

The remainder of this paper is organized as follows: Sections 2 and 3 summarize the necessary background and related GNN-based IDS approaches. Section 4 presents the motivation and problem formulation, while Section 5 describes the proposed method. Section 6 reports the experimental results, followed by a representative case study in Section 7. Section 8 discusses the main limitations, and Section 9 concludes the paper with future research directions.

2 Background

2.1 Network intrusion detection systems

Intrusion Detection Systems (IDS) are fundamental components of modern cybersecurity architectures. Their primary objective is to monitor system activities (e.g., network traffic, system logs, resource access, etc.) to identify malicious behavior patterns or policy violations. By detecting suspicious activity, IDS provide visibility into threats that may bypass preventive defenses, thereby supporting both incident response and forensic analysis.[15]

IDS are commonly divided into two major categories. Network-based IDS (NIDS) analyze network traffic to detect anomalies or known attack signatures, while host-based IDS (HIDS) focus on monitoring events, configurations, and system calls on individual machines.[16] Both types play a crucial role within a Defense-in-Depth strategy, where multiple layers of security controls are combined to ensure resilience against threats.[17]

A closely related concept is the Intrusion Prevention System (IPS). While IDS are primarily designed for detection and alerting, IPS solutions extend this capability by actively blocking or mitigating malicious traffic in real time. This distinction highlights their complementary nature: IDS emphasizes detection and situational awareness, whereas

IPS emphasizes prevention and enforcement. In practice, hybrid IDS/IPS solutions are increasingly common, though the conceptual difference remains relevant in research and deployment scenarios.

In specialized environments such as the Internet of Things (IoT), intrusion detection introduces additional challenges. IoT devices are often resource-constrained, heterogeneous, and deployed at scale, so traditional IDS techniques are often impractical for examining these systems. Heterogeneous protocols demand adaptable detection methods, while the distributed nature of IoT calls for scalable monitoring solutions.[18,19] Machine learning-based anomaly detection is increasingly applied to handle evolving IoT traffic patterns.[19]

Overall, IDS's role is to enhance situational awareness, complement preventive defenses, and provide an essential detection layer in modern cybersecurity ecosystems.

2.2 Graph neural networks

Graph Neural Networks (GNNs) are deep learning architectures, that provide a framework for representation learning of graph-structured data, where the nodes represent entities and the edges represent relations between them.[20] Unlike traditional neural networks, which assume Euclidean structure in the input (e.g., images, time-series, natural language sentences), GNNs operate directly on non-Euclidean domains, making them particularly suitable in various application fields (e.g., chemistry, social networks, knowledge graphs, distributed systems).[21] A GNN operates by iteratively updating the representation of each node based on the features of its neighbors and the graph structure. The node representation update typically follows a message-passing paradigm, formalized by three key operators: AGGREGATE, COMBINE, and READOUT.

2.2.1 Message passing. The following formulation summarizes the fundamental message-passing procedure used in graph neural networks. For a graph $G = (V, E)$, where V is the set of nodes, and $E \subseteq V \times V$ is the set of edges. Each node $v \in V$ is associated with a feature vector $h_v^{(0)} \in \mathbb{R}^d$, which serves as the input to the first GNN layer. The node representations are updated layer by layer over L iterations, where each layer $l \in \{1, \ldots, L\}$ performs an AGGREGATE-COMBINE step, based on the output of the previous layer.[22]

In the AGGREGATE step, for each node v, messages from the neighboring nodes $\mathcal{N}(v)$ are aggregated:

$$m_v^{(l)} = \text{AGGREGATE}^{(l)}(\{h_u^{(l-1)} : u \in \mathcal{N}(v)\}). \tag{1}$$

The aggregation function is typically permutation-invariant, ensuring independence from the order of neighbors. In the COMBINE step, the node's own representation is updated by combining it with the aggregated message:

$$h_v^{(l)} = \text{COMBINE}^{(l)}(h_v^{(l-1)}, m_v^{(l)}). \tag{2}$$

In practice, this is often implemented as a linear transformation followed by a nonlinearity:

$$h_v^{(l)} = \sigma(W^{(l)} \cdot [h_v^{(l-1)} || m_v^{(l)}]), \tag{3}$$

where $||$ denotes concatenation, $W^{(l)}$ is a learnable weight matrix, and σ is an activation function such as ReLU. For graph representation tasks, the READOUT function is designed to aggregate features of distinct nodes to the representation of the entire graph as a final step of the encoding:

$$h_G = \text{READOUT}(\{h_v^{(L)} : v \in V\}) \tag{4}$$

The READOUT function is often permutation-invariant (e.g., max, sum, mean) or attention-based pooling over the final node embeddings.[23] The resulting graph representation can then be used to extract information using a downstream model (e.g., an MLP or a Random Forest).

2.2.2 Graph attention networks. Graph Attention Networks (GATs)[24] extend the message-passing framework of GNNs by incorporating an attention mechanism to assign importance to each neighbor when aggregating node features. Unlike earlier GNN variants such as GCN,[20] where neighbors contribute equally or based on fixed normalization, GATs learn to weigh neighbors dynamically based on their relevance.

First, each node's feature vector is transformed using a learnable matrix:

$$h_v' = W \cdot h_v, \tag{5}$$

where h_v is the original feature vector of node v, and W is a weight matrix learned during training. For each pair of connected nodes v and u, the model computes an attention score e_{vu}, which measures how important node u is to node v:

$$e_{vu} = \text{LeakyReLU}(a^T[h'_v||h'_u]) \tag{6}$$

Here, a is a vector of learnable parameters, and $[h'_v||h'_u]$ means the concatenation of the features of the two nodes. The attention scores are normalized over all neighbors of node v:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k\in\mathcal{N}(v)}\exp(e_{vk})} \tag{7}$$

This gives the attention weights α_{vu}, telling the model how much attention node v should pay to node u. Finally, the updated feature of node v is computed by taking a weighted sum of its neighbors' features:

$$h''_v = \sum_{u\in\mathcal{N}(v)} \alpha_{vu} \cdot h'_u \tag{8}$$

This means node v gets information from its neighbors, but more from the ones it pays more attention to.

2.2.3 Edge-aware graph neural networks. Most standard GNN architectures primarily focus on learning from node features and graph structure, while often neglecting or inadequately utilizing edge features.[25,26] However, in many real-world applications (e.g., network traffic analysis, molecular interaction modeling, or transportation networks), important information is associated with the edges themselves. For example, in network communication graphs, performance metrics such as latency, bandwidth, and packet loss are naturally associated with edges rather than nodes.[6]

To overcome this limitation, two directions have emerged. One approach is to transform the original graph into its *line graph*,[27] where edges become nodes. For a graph $G = (V, E)$, its line graph $L(G)$ is the graph whose nodes correspond to the edges of G, and where two nodes are adjacent if and only if their corresponding edges in G share a common endpoint. This allows standard GNNs to process edge information within their existing message-passing framework, while the results can still be mapped back to the original graph structure. However, constructing the line graph often results in a significant increase in graph size, especially for high-degree nodes, making the approach computationally expensive. Moreover, the transformation may obscure the original topology and does not always provide intuitive representation of the input graph.

The other approach is to design edge-aware GNN architectures that explicitly incorporate edge attributes into the message-passing process, for example by conditioning aggregation mechanisms on edge features, or by learning joint representations of node pairs and their connecting edges. These approaches include *R-GCN*,[28] which is specifically tailored for the processing of multi-relational data, employing relation-specific transformations for aggregating the features of neighboring nodes and leveraging parameter-sharing techniques to enable the efficient modeling of relations. Furthermore *EdgeConv*,[29] a neural network algorithm that leverages both global structure and local neighborhood information.[30] Introduces the extension of the *Graph Isomorphism Network* to handle edge features. *EdgeGAT*[31] is an extension of the Graph Attention Network, that incorporates edge features into its attention mechanism, thereby enabling more expressive modeling of interactions between nodes.

2.3 Architectural requirements of GNN-based IDS

IDS operate under several constraints that must be reflected in the design of detection models. Since network traffic is continuous, anomalies must be detected during system operation rather than through offline analysis. Practical IDS deployment also requires low-latency processing, meaning that decisions often need to be made from short observation windows to enable timely mitigation of attacks. Another challenge arises from the dynamic nature of network communication, where hosts and connections continuously appear and disappear over time. Furthermore, important communication semantics are typically associated with packet flows rather than individual hosts, while evolving traffic characteristics and attack strategies introduce concept drift that may gradually degrade detection performance. In practical environments, explainability is also essential, as administrators require interpretable evidence to support remediation procedures and incident analysis.

These requirements make GNNs a particularly suitable foundation for modern IDS. Network communication can naturally be represented as a graph, where hosts or sockets correspond to nodes and communication flows correspond to edges. GNNs are specifically designed to learn representations from such graph-structured data and to capture topological attack

patterns through message passing. Edge-aware GNN architectures can incorporate flow-level semantics directly into the learning process, while inductive GNN models can generalize to previously unseen hosts, connections, and graph structures. Furthermore, attention-based GNNs, such as GAT variants, can support intrinsic explainability by highlighting the communications most relevant to a prediction, thereby helping administrators focus on evidence that may be useful for remediation.

Attention-based edge-aware architectures also provide a mechanism for reducing the influence of irrelevant background communication. Since attention weights are learned from both communication structure and flow-level characteristics, less informative flows may receive lower importance during representation learning, while communication patterns that are more relevant to the prediction can contribute more strongly. This is particularly important in realistic traffic, where benign background communication may dominate the observed flows. Moreover, combining flow-level attributes with graph topology allows the model to exploit complementary information sources rather than relying solely on individual traffic characteristics. The use of temporally ordered evaluation further provides a more realistic assessment under evolving traffic conditions, since predictions are evaluated on traffic that occurs after the training samples.

These considerations directly influenced the design of the proposed framework. Continuous monitoring and low-latency requirements motivated the use of short time-windowed traffic graphs, while the flow-centric nature of network communication motivated the adoption of an edge-aware GNN architecture. Furthermore, the need for explainability and robustness against evolving communication patterns motivated the use of an attention-based inductive GNN model.

3 Related work

Traditional machine learning methods have been widely explored for intrusion detection in distributed environments. Churcher et al.[32] evaluated several classical ML classifiers, showing strong performance for random-forest in binary classification and KNN in multiclass classification. Altunay et al.[33] investigated CNN, LSTM, and hybrid IDS models for IIoT networks, demonstrating the effectiveness of deep learning architectures in capturing complex traffic patterns. Parfenov et al.[34] examined adversarial data leakage attacks against conventional ML-based models, highlighting robustness concerns in realistic cloud-based deployments.

Recent studies have investigated GNN-based IDS approaches, showing potential to surpass traditional techniques.

Some models are designed for specific attack scenarios rather than general-purpose intrusion detection. *GraphDDoS*[12] converts network traffic into endpoint traffic graphs and captures both packet-level relationships within individual flows, such as SYN-flood-related structures, and flow-level relationships, including multi-flow burst and periodic patterns. A GNN classifier then learns traffic patterns characteristic of DDoS attacks. Similarly, Zhang et al.[35] proposed a graph embedding-based method for IoT botnet detection, focusing on identifying P2P botnet nodes in large-scale communication graphs. Their approach exploits the topological characteristics of botnet communication. Both methods address specialized detection tasks and operate on large, aggregated traffic graphs, making them more suitable for retrospective analysis than low-latency online monitoring. In contrast, our method targets general-purpose, multi-class intrusion detection by classifying short time-windowed traffic graphs in an online manner.

Friji, et al.[36] propose a flow-based graph representation in which flows are modeled as nodes and edges connect related flows, enabling the detection of iterative malicious behavior, IP spoofing activity, and multi-step attacks. Their framework combines MLP-based embedding, attention-based feature extraction, and GCN-based spatial feature extraction to assign maliciousness scores to individual flows. While this design is effective for flow-level intrusion detection, it introduces a relatively complex multi-module architecture. In contrast, our method focuses on online graph-level classification over short traffic windows and uses a simpler edge-aware attention model, from which interpretation subgraphs can be derived directly.

E-GraphSAGE[7] is an extension of the widely utilized *GraphSAGE*[37] architecture, which explicitly incorporates edge features into the message-passing process. As an intrusion detection framework, it represents communication endpoints as nodes and network flows as edges, enabling edge classification on large-scale IoT traffic graphs by leveraging both topological structure and flow attributes. However, its GraphSAGE-based aggregation relies on predefined neighborhood aggregation functions, whereas attention-based models can learn the relative importance of neighboring communications dynamically. Previous studies also observed that graph propagation may weaken the contribution of individual node attributes during message passing.[38] In contrast, our method adopts an edge-aware GAT architecture, which can dynamically learn the relative importance of neighboring communications, emphasize the most relevant traffic patterns, and provide intrinsic support for interpretation through attention scores. E-GraphSAGE is a widely cited baseline, that has inspired numerous subsequent studies.[39] Many works extend its abilities with, e.g., residual learning,[10] self-supervised learning,[40] and encodings of certain centrality measures.[41]

EMA-IDS[42] combines node and edge information and evaluates GNN-based intrusion detection on large dataset-level communication graphs and applies mini-batch subgraph sampling during training. However, the authors explicitly identify the temporality of attacks as an aspect not addressed in their study.

Although these methods demonstrate the potential of GNNs for intrusion detection, their practical deployment remains challenging. Many of them operate on large, preconstructed traffic graphs collected over days or weeks, which assumes that a detailed traffic graph is already available before prediction. This setting is suitable for offline analysis, but less aligned with operational IDS scenarios, where traffic arrives continuously and decisions must be made from partial, time-constrained observations. GNN-based IDS approaches operating on temporally bounded traffic data remain underrepresented in the literature, although a few preliminary works have addressed this direction.[14,43] Stream-based intrusion detection has also been investigated in prior work,[44,45] however, such approaches generally do not exploit graph-based traffic representations.

TE-G-SAGE[46] uses a chronological train-validation-test split to preserve temporal ordering. However, unlike our approach, their evaluation was conducted on a large temporal graph spanning an extended observation period, rather than on periodically constructed graph samples based on fixed-duration traffic windows. DIGNN-A[47] also preserves the chronological order of data flows; however, rather than constructing traffic graphs over predefined time intervals, it constructs them after collecting a fixed number of flows. Consequently, lower traffic intensity may result in higher detection latency.

Venturi, et al.[14] evaluate the practical applicability of GNN-based IDS by adapting E-GraphSAGE to a more realistic time-windowed setting. Instead of using complete test graphs, they construct separate evaluation graphs from fixed-length traffic windows, showing that shorter windows can substantially reduce detection performance and reveal a trade-off between latency and accuracy. Their approach performs E-GraphSAGE-based flow-level classification and primarily focuses on how window size affects detection performance, without addressing explanation generation or computational complexity in detail, even though both are important aspects of practical IDS deployment.

PPT-GNN[43] addresses practical limitations of earlier GNN-based IDS methods by using short sliding time windows, spatio-temporal graph construction, and self-supervised pre-training. It models spatial relations between IP and flow nodes, and temporal dependencies across consecutive windows, enabling near-real-time flow-level classification and improved transferability across network environments. However, its graph construction operates at the IP level and does not explicitly distinguish between different services or sockets running on the same host. In contrast, our method uses a finer-grained IP:port-based traffic graph and integrates attention-based explanation to support human analysis.

Based on recent survey,[5] the main open challenges in GNN-based intrusion detection concern graph construction, model design, and practical deployment. First, network traffic does not inherently appear as graph-structured data; therefore, the choice of nodes, edges, and temporal segmentation strongly affects what attack patterns can be learned. Second, most existing methods focus on node- or edge-level classification, while graph-level detection remains relatively scarce, despite its relevance for assessing the overall state of a monitored system. Third, dynamic graph settings remain challenging, because IDS models must process continuously evolving traffic under strict latency and resource constraints. Finally, explainability remains insufficiently addressed, although security operators require understandable evidence behind alerts to support incident response.

Our method addresses several of these gaps by constructing fine-grained IP:port-based traffic graphs from short time windows, performing online graph-level classification, and generating attention-based explanatory subgraphs. In this way, the proposed approach focuses on bridging the practical gap between accurate graph-based detection, timely online operation, and human-interpretable alerting.

4 Motivation and problem statement

4.1 Motivation

AI-based intrusion detection systems (IDS) adopt a wide variety of approaches. In the last decade, deep learning has gained prominence, with models such as LSTM networks, Transformers, which methods leverage temporal properties of network traffic to detect complex multi-step attacks. In recent years, several Graph Neural Network (GNN)-based IDS models have been proposed, and studies have reported that they outperform sequence-based methods in many intrusion detection scenarios.

However, the evaluation methodology adopted by most existing GNN-based IDS approaches has been criticised.[14] In many studies, training and testing are conducted on network traffic graphs that aggregate operational characteristics over long periods, often spanning several days or weeks.[7,10,13,48] More importantly, the split between training and testing edges

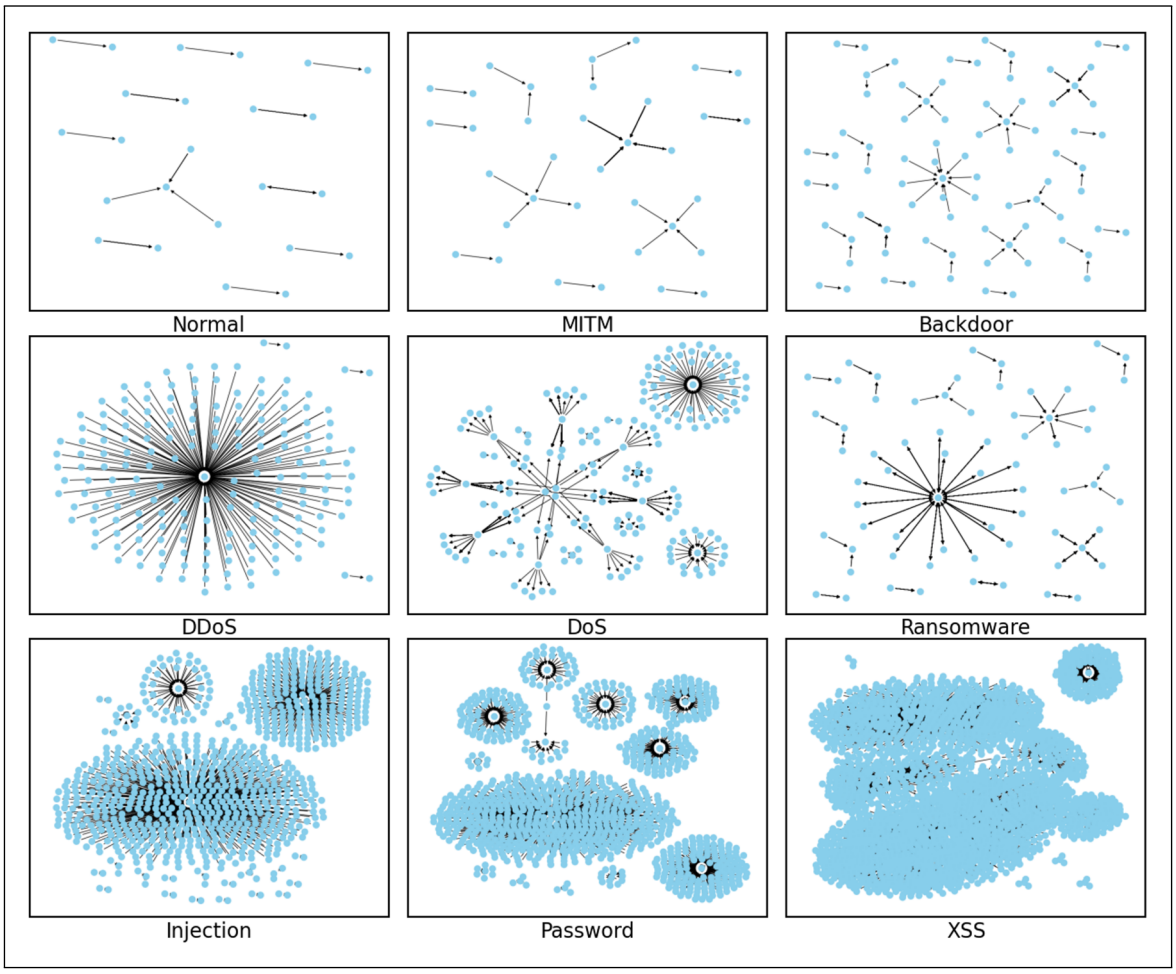


Figure 1. Network traffic graphs of normal operation and different attacks in ToN-IoT dataset.

is often performed randomly within the graph, rather than along temporal boundaries. This practice risks information leakage, since structural or temporal cues from future traffic may inadvertently influence the training process. As a result, the model can exploit patterns that would not realistically be available in a real-world operation scenario. Consequently, these methods tend to overestimate detection performance, since in practice it is unknown when a graph should be constructed, moreover predictions must be made continuously on partial, time-constrained data. This gap highlights a critical trade-off: while large, aggregated graphs yield higher accuracy, they introduce unacceptable detection delays; conversely, shorter time windows reduce latency but may degrade performance significantly. Such discrepancies underscore the challenges of translating GNN-based IDS research results into practical online deployment.

As an initial step to address these gaps, we aimed to explore whether network traffic graphs constructed from shorter time windows could reveal visually distinguishable patterns between normal operation and different attack types. This exploratory analysis was conducted by segmenting the ToN-IoT Network dataset[49] into 20-s windows and randomly selecting representative graphs for both normal traffic and attack scenarios. The visual outcomes, shown in Figure 1, served as a preliminary observation that motivated our subsequent analysis. In these graphs, nodes represent unique IP:port pairs (i.e., network sockets), while edges denote the communication between them.

The individual network traffic graphs appear to be relatively well distinguishable from one another. In normal traffic, the graphs consist primarily of small two-node components, reflecting balanced communication in which no single host dominates. By contrast, *man-in-the-middle* attack exhibit intermediate nodes inserted between communicating peers. These nodes act as relays, intercepting and forwarding traffic, which structurally embodies the essence of eavesdropping or message manipulation. Graph corresponding to *backdoor* attacks frequently display isolated star-shaped subgraphs in which a single node communicates asymmetrically with multiple others. Such hubs may represent hidden control channels or covert access points that bypass standard authentication mechanisms. In *ransomware* attack graph, one node often shows

high-degree bidirectional communication with surrounding nodes, indicative of both infection spread and status propagation. The presence of repeated small-scale clusters further supports the interpretation of a rapidly expanding infection pattern. *DDoS* graph reveals a highly centralized structure, typically with a single target node receiving incoming connections from many distributed nodes. This dense radial topology mirrors the large-scale, simultaneous nature of DDoS attacks, where numerous compromised hosts flood a single victim with traffic. In contrast, *DoS* graph consists of multiple local clusters where a few nodes act as hubs for outward communication. These smaller, repeated star-like patterns suggest single-source flooding aimed at different targets, consistent with localized service disruption. *Injection* attack graph reveals attacker nodes initiating burst-like outbound connections toward many destinations, reflecting automated exploitation attempts such as SQL or client-side injections against vulnerable web applications. *XSS* graph can be characterized by densely clustered substructures with complex edge configurations, indicative of malicious script propagation in interactive web environments through client-side code injection. Finally, *password* attack graph is marked by recurring star-like motifs in which single nodes issue large volumes of login attempts, consistent with brute-force or dictionary-based password cracking activities.

Our exploratory qualitative analysis suggests that structural information is likely to play a significant role in detecting network anomalies and in distinguishing between different attack types, even when traffic from shorter time windows is available for the IDS.

These observations indicated the feasibility of developing an IDS that performs online classification of network traffic monitoring data to enable early detection of system intrusions. In the event of a detected anomaly, such IDS can activate packet capturing for fine-grained traffic inspection and/or notify human operators about the suspicion of an ongoing attack to the system. Importantly, these alerts can be accompanied by explanatory information that identifies the most relevant network communications underlying the IDS decision. By combining rapid detection with transparent explanations, the system supports administrators in promptly triaging, investigating, and mitigating attacks. While our focus in this study remains on intrusion detection, the proposed framework could be extended toward an Intrusion Prevention System (IPS), enabling the automatic utilization of countermeasures such as blocking suspicious IP addresses, updating firewall rules, throttling malicious traffic, or forwarding alerts to external incident management systems.

4.2 Problem formulation

Given network traffic observed over continuous time, we consider fixed-length time windows of size $\tau > 0$. For a window starting at time t, let

$$T_t = [t,\ t + \tau). \tag{9}$$

Within T_t, communication occurs between network sockets (often corresponding to IP:port pairs used to communicate through UDP or TCP sessions). We build a directed multigraph

$$G_t = (V_t,\ E_t,\ X_t), \tag{10}$$

where each node $v \in V_t$ represents a unique socket observed in T_t, and the edge set is defined as $E_t \subseteq V_t \times V_t \times \mathbb{N}$. Each edge $e \in E_t$ is a triplet $e = (u,\ v,\ i)$, representing the i -th communication flow from node u to node v within T_t. Each edge e carries its own d -dimensional feature vector $x_e \in \mathbb{R}^d$, which describes the characteristics of the communication flow. Let X_t denote all x_e recorded within T_t. Let the set of classes be $Y = \{0,\ 1,\ \ldots,\ c\}$, where 0 denotes normal operation and $1,\ \ldots,\ c$ denote distinct attack types. The primary goal is to find a mapping

$$M : (V_t,\ E_t,\ X_t) \to Y, \tag{11}$$

that outputs a class $y \in Y$ using data available within T_t.

The secondary goal is to find an explanation in the form of a subgraph

$$G'_t = (V'_t,\ E'_t,\ X'_t), \tag{12}$$

where $V'_t \subseteq V_t$ and $E'_t \subseteq E_t \cap (V'_t \times V'_t \times \mathbb{N})$ and $X'_t = \{x_e | e \in E'_t\}$, which highlights the most relevant part of the traffic graph that contributed to the model's decision.

5 Approach

Our proposed IDS is depicted in Figure 2. It continuously monitors network traffic and, at predefined intervals, performs analysis to generate alerts. Each alert conveys the system's current operational state, explicitly indicating whether the

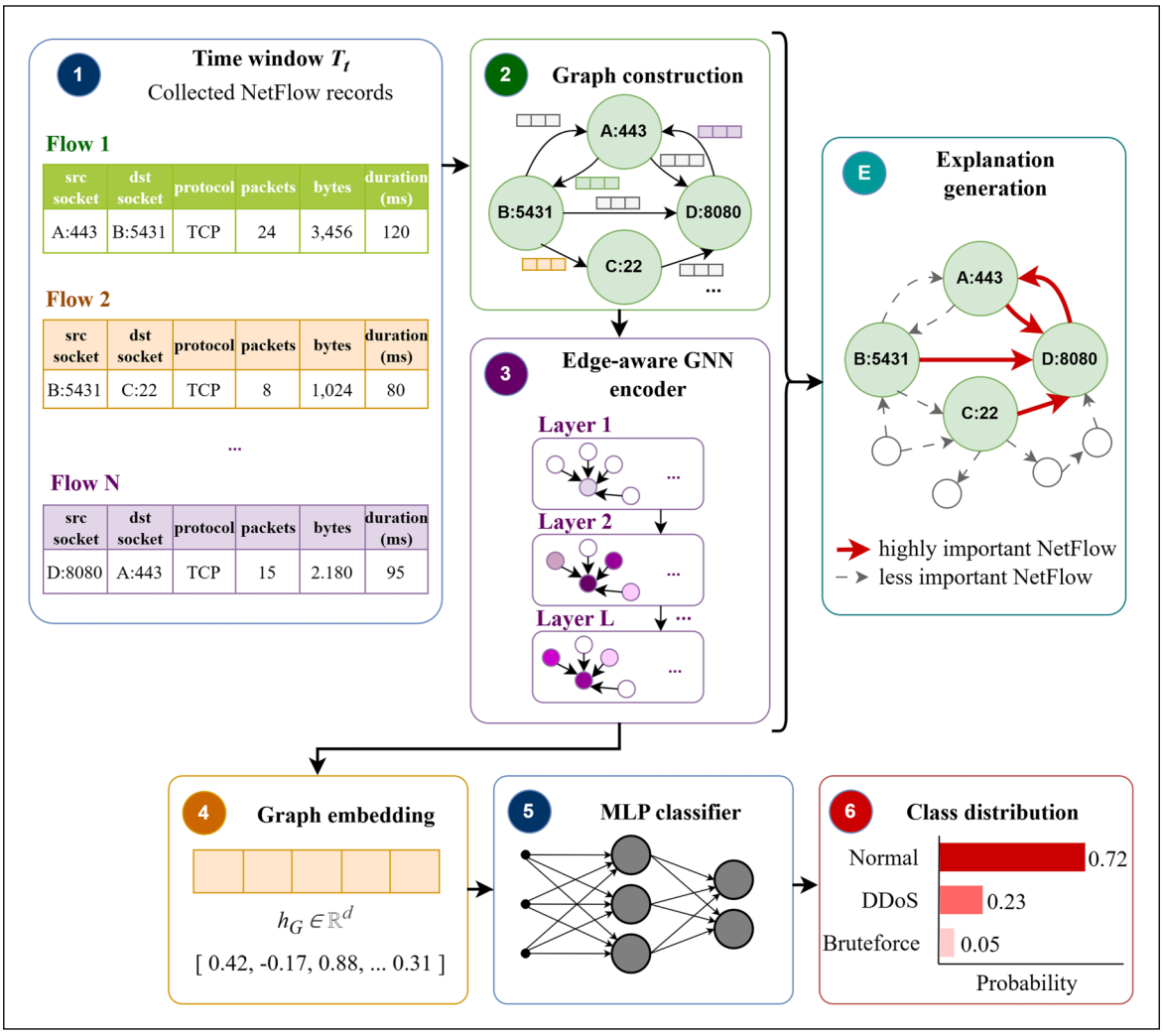


Figure 2. Architecture of the proposed IDS method.

network is functioning normally or a specific intrusion is underway. In addition, the alert includes an explanatory subgraph that highlights the most relevant communications underlying the detection. Furthermore, alerts can be forwarded to an Intrusion Prevention System (IPS), which may conduct fine-grained monitoring of the affected system component or automatically initiate defensive countermeasures.

Figure 3 illustrates the operation of the proposed detection method, it depicts the onset of a DDoS attack. During the first time interval, the system operates normally, with communication primarily consisting of point-to-point connections. In the second interval, the early stage of a DDoS attack emerges. As time progresses, the attack may expand further, increasingly disrupting the system's normal operation. From time 0 onward, the system is continuously monitored, and network communication flows within the system are recorded. At each time step t, a traffic graph is constructed from the flows observed up to that point. This graph is then encoded by a Graph Neural Network (GNN), and the resulting representation is passed to a multilayer perceptron (MLP) that predicts the type of attack - if any - that may currently be affecting the system.

5.1 Graph construction

Graph construction has a substantial impact on the representational quality and detection performance of GNN-based IDS models. In our approach, network traffic is represented as a directed multigraph in which nodes correspond to unique IP:port pairs, and edges represent communication flows observed between them within a given time window. Each edge is annotated with an $x_e \in \mathbb{R}^d$ feature vector of dimension d, that captures statistical and protocol-level descriptors of

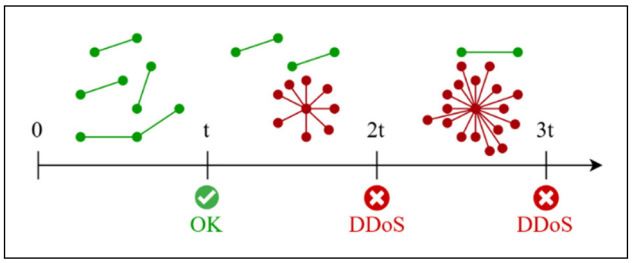


Figure 3. Time-window-based online intrusion detection with traffic graph construction.

the underlying flow (e.g., number of packets, flow duration, TCP flags, average of inter-packet arrival time). The IP:port identifiers are used exclusively for constructing the graph structure; they are not provided to the model during training. This restriction prevents the model from relying on spurious correlations, such as associating specific IP addresses with attacks, which would constitute a form of data leakage and severely undermine generalization, since in many network intrusion datasets only a small subset of IP addresses generate malicious traffic. Following recommendations of,[7] node features are initialized as $h_v^{(0)} \in \mathbf{1}_d$ all-one vectors of the same dimensionality as the edge features, ensuring structural consistency at the node level.

These design decisions are justified by considering the trade-off between granularity and scalability. Compared with graphs constructed at the IP level only, the inclusion of port numbers enables a more fine-grained representation of the system's communication structure. Distinct services hosted on the same IP address (e.g., HTTPS on port 443 and SSH on port 22) can be differentiated, enabling the model to identify attack patterns targeting specific services that would otherwise be indistinguishable. This finer granularity is particularly valuable in environments where multiple applications coexist on the same host.

Furthermore, we adopt a flow-based representation for edges, rather than packet-based. Flow aggregation provides a natural unit of communication, capturing the essential properties of socket interactions while avoiding the excessive computational and storage overhead of per-packet processing. Organizations typically activate packet-level monitoring only for forensic investigations, as it entails maintaining very large volumes of detailed data and hinders scalability in real-time intrusion detection. By contrast, flow-based graphs strike a balance between granularity and scalability, retaining the structural and statistical characteristics of traffic necessary for anomaly detection, while keeping graph sizes manageable for online processing.

5.2 Representation learning

The proposed method follows an encoder–decoder architecture. During encoding, the objective is to extract informative representations from the traffic graph constructed in the previous step, achieved through graph representation learning with an edge-aware GAT.

During encoding, one or more GNN layers are utilized to produce a fixed-size numeric representation of the input graph. For this purpose, we adopt the *EdgeGAT* architecture proposed by.[31] Unlike standard GNNs, it explicitly incorporates edge features, allowing the model to jointly exploit node characteristics, edge features, and graph topology. This is advantageous in our setting, where flow-level information is associated with edges. The use of attention further enhances the model's suitability for intrusion detection, as it enables the network to learn which communication flows are most relevant for distinguishing normal from anomalous behavior. Moreover, attention-based architectures are inherently explainable: the learned attention scores directly indicate the relative importance of edges in the decision process, thus providing a basis for generating explanatory subgraphs. Formally, the update of the features of a node v is defined by

$$h_v^{'} = W_s \cdot h_v + \sum_{u \in \mathcal{N}(v)} \alpha_{uv}(W_n \cdot h_u + W_e \cdot x_{uv}), \tag{13}$$

where $\mathcal{N}(v)$ stands for the neighbors of node v, and W denotes learnable weight matrices with different roles: W_s is used for the residual/self-update term, W_n projects node features, and W_e projects edge features to the attention space.[31] Attention scores are calculated using

$$\alpha_{uv} = softmax_v(LeakyReLU(a^T[W_n \cdot h_v || W_n \cdot h_u || W_e \cdot x_{uv}])), \tag{14}$$

with a corresponding to a learnable vector, and $softmax_v$ standing for the normalization by all incoming edges of node v.

To obtain a graph-level representation, the node embeddings are aggregated into a fixed-size vector by max pooling, providing a compact summary of the traffic graph for downstream classification.

This representation is then passed to a multilayer perceptron (MLP) that performs graph-level classification. We adopt an MLP decoder due to its simplicity and effectiveness, as it is a standard baseline for processing graph embeddings. It outputs the predicted label indicating whether the system is operating normally or is subject to a specific type of attack.

5.3 Explanation generation

After classification, a subgraph of the complete traffic graph is generated to serve as an explanation of the decision. This subgraph can assist administrators in understanding why the IDS raised a suspicion of an attack and provide guidance for identifying the potential root causes of the intrusion. The objective is therefore to construct a traffic subgraph that represents the network activity observed within the given time window in the most compact and informative manner possible. Importantly, the explanation subgraph does not necessarily consist solely of edges directly associated with malicious activity. Certain patterns of normal communication may also be included, to provide context for distinguishing abnormal traffic from legitimate background activity.

To generate explanations for model decisions, we aggregate the attention scores assigned to edges across multiple GNN layers and encoder attention heads. This is performed as a bottom-up hierarchical aggregation, in which attention values are first reduced at the head level within each layer, and subsequently across layers to obtain a single importance score per edge. The R_{head} and R_{layer} operators must be permutation-invariant (e.g., mean, max, sum) to ensure robustness with respect to the ordering of heads and layers. Different operators highlight different explanatory perspectives: e.g., the mean emphasizes edges that are consistently deemed important across heads and layers, whereas the max highlights edges that were considered highly relevant in at least one head or layer. Based on the aggregated scores, we retain the edges belonging to the top- p percentile together with their adjacent nodes, where p is a configurable hyperparameter controlling the sparsity of the explanation graph. Using a percentile-based threshold, rather than a fixed top-k selection, ensures that edges with identical importance scores at the cutoff are consistently preserved in the explanation. The resulting subgraph represents the most relevant part of the traffic graph, serving as the explanation of the model's decision.

This process is formalized as Algorithm 1, where $\alpha \in \mathbb{R}^{|E|\times L\times H}$ contains normalized edgewise attention scores produced by the encoder. L is the number of encoder layers and H is the number of attention heads per layer. For an edge $e \in E$, layer $\ell \in 1, \ldots, L$, and head $h \in 1, \ldots, H$, $\alpha_{e,\ell,h}$ denotes the scalar attention score.

In Lines 1–4 the algorithm performs the head-level reduction: for each edge and each layer, the attention scores across all heads are aggregated, producing one value per edge per layer. In Line 5 the layer-level reduction is carried out, combining the per-layer values into a single importance score for each edge. In Line 7 the percentile-based threshold is calculated, determining the cutoff value corresponding to the top- p percentile of edge importance scores. In Line 8 the set of important edges is selected by retaining those whose importance score is greater than or equal to the threshold. In Line 9 the corresponding set of nodes is determined by including every node that is connected to at least one of the retained edges. Finally, in Line 10 the explanatory subgraph is returned, representing the most relevant part of the traffic graph in the model's decision.

6 Evaluation

In this section, we evaluate the performance of our approach by answering the following Research Questions:

- **RQ1**: How effectively does the model detect intrusions?
- **RQ2**: How sensitive is the model to the chosen time window size?
- **RQ3**: How sensitive is the model to its hyperparameters?
- **RQ4**: How explainable are the model's predictions?

6.1 Datasets

To evaluate our approach, we selected two representative network traffic datasets, each containing normal communication and multiple attack patterns.

D1. The *ToN-IoT Network* dataset[49] was collected on a realistic large-scale testbed that integrates edge, fog, and cloud layers, capturing heterogeneous data sources from IoT network traffic. It includes nine contemporary cyberattack scenarios. This heterogeneity makes the dataset valuable for evaluating our approach in complex and distributed environments.

Algorithm 1. Extraction of the explanatory subgraph.

Input:

$G(V, E)$: traffic graph for a given time window,

α: attention scores,

R_{head}, R_{layer}: reduction operators

p: sparsity parameter.

Output:

G': explanation subgraph.

Procedure:

for $e = 1$ **to** $|E|$ **do**
  for $\ell = 1$ **to** L **do**
    $\alpha_{layer}[e, \ell] \leftarrow R_{head}(\{\alpha_{e,\ell,h} : h = 1..H\})$
  end for
  $importance[e] \leftarrow R_{layer}(\{\alpha_{layer}[e, \ell] : \ell = 1..L\})$
end for
$\tau \leftarrow Quantile_{1-p}(\{importance[e] : e = 1..|E|\})$
$E' \leftarrow \{e \in E : importance[e] \geq \tau\}$
$V' \leftarrow \{u \in V : \exists (u, v) \in E' \text{ or } (v, u) \in E'\}$
$G' \leftarrow (V', E')$

Table 1. Main characteristics of the used datasets.

Dataset	Packet flows	Normal %	No. classes	No. raw features
D1	22,339,021	3.56	10	40
D2	20,115,529	87.07	7	47

D2. The *NF-CSE-CIC-IDS2018-v3* dataset[50] is a NetFlow v3 conversion of the CSE-CIC-IDS2018[51] benchmark, enriched with temporal features. It contains flow records collected from a realistic network environment, covering diverse attack categories. By preserving temporal dynamics, the dataset enables analysis of time-dependent traffic patterns and the progression of complex attack scenarios.

Table 1 presents the key characteristics of the datasets used. Each dataset comprises 20–22 million unique packet flows, yet they differ substantially in their statistical distributions. In D1, only 3.56% of the traffic is benign, while 9 distinct attack classes are represented. The strong prevalence of attack scenarios allows for examining the model's ability to effectively differentiate between a diverse set of malicious behaviors. In contrast, D2 more closely resembles real-world conditions, with only 12.93% of the traffic associated with network attacks. Moreover, this dataset incorporates NetFlow v3 features, including temporal statistics such as inter-packet arrival times, which provide a more comprehensive representation of network dynamics. Note that the reported "No. Raw Features" refers to the attributes available for edge annotation; identifiers such as source/destination IP and port are excluded.

Figure 4 summarizes class distributions among the datasets. The datasets exhibit notably different distributions: D1 contains only a small amount of normal traffic and multiple attack classes, yielding an attack-rich, imbalanced setting. By contrast, D2 is dominated by normal traffic and contains comparatively smaller attack proportions. These complementary skews enable a robust assessment of the model's generalizability across attack-heavy and more realistic operating conditions.

6.2 Data preprocessing

The preprocessing pipeline was designed to transform the raw datasets into a structured form suitable for graph-based anomaly detection. First, flows were ordered chronologically and segmented into 20-s windows. Then, auxiliary or unused attributes (e.g., URIs, SSL certificate identifiers, user agents) were discarded, resulting in 31 utilizable edge features for D1 and 46 for D2. To ensure consistency, invalid numerical entries (e.g., NaN, Inf) were checked, but none were found. Next, source and destination sockets were constructed by concatenating IP addresses with port numbers. To avoid spurious correlations, these identifiers were later used solely for graph topology construction, and their actual values were not included as features in the IDS graph representation.

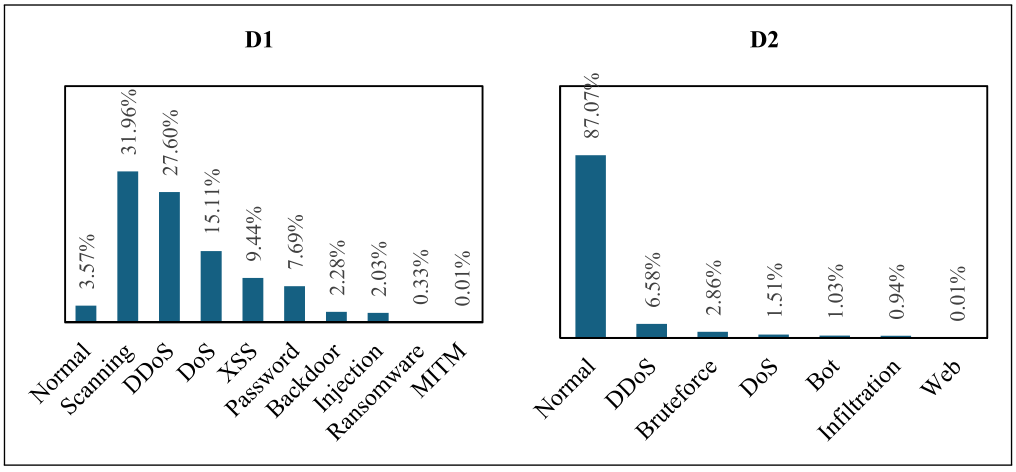


Figure 4. Class distributions in the used datasets.

Time windows were labeled based on the flow-level annotations provided in the datasets. Windows containing only normal flows were assigned the normal label, while those including normal traffic together with a single attack type were labeled according to that attack. Windows in which flows corresponding to multiple attack types co-occurred were excluded from both training and evaluation. Such cases were rare and occurred only in D2.

The train–test split was performed to preserve the overall distribution of the datasets. For each label, the first 80% of the time windows were assigned to the training set, and the remaining 20% to the test set. Finally, categorical variables (e.g., protocol, service, connection state attributes) were encoded using target encoding, and numerical features were standardized via z-score normalization, both fitted only on the training set to prevent data leakage.

6.3 Metrics

To evaluate the performance of our approach, we utilized the widely known $precision = \frac{TP}{TP+FP}$, , $recall = \frac{TP}{TP+FN}$, and $F1 = 2 \cdot \frac{precision \cdot recall}{precision+recall}$ metrics. Here, *TP* are instances correctly predicted as the class of interest, *FP* are instances incorrectly predicted as that class, *FN* are instances of that class incorrectly predicted as another class, and *TN* are all other correctly identified instances not belonging to the class. The final reported values are obtained by applying a weighted average across all classes, where the weights correspond to the class frequencies in the dataset.

As a second objective, an explanation subgraph is generated. To quantitatively evaluate the quality of these explanations, we rely on three commonly used explainability metrics ($fidelity_-$, $fidelity_+$, and *sparsity*), and introduce a domain-specific measure, *edge ratio*. These metrics are calculated per class and aggregated using class-frequency-weighted averages.

$Fidelity_-$ measures the agreement between the prediction of the explanatory subgraph and that of the original full graph. Formally, for a set of graphs $\mathcal{G}$ with predictions $f(\cdot)$, and explanation subgraphs $G' \subseteq G$,

$$Fidelity_- = \frac{1}{|\mathcal{G}|} \sum_{G \in \mathcal{G}} \mathbf{1}[f(G') = f(G)], \tag{15}$$

where $\mathbf{1}[\cdot]$ is the indicator function. A value of 1 indicates perfect consistency.

$Fidelity_+$ quantifies how often the removal of the explanation subgraph alters the model's prediction. For each graph G, let $G \setminus G'$ denote the residual graph after removing the explanation. Then,

$$Fidelity_+ = \frac{1}{|\mathcal{G}|} \sum_{G \in \mathcal{G}} \mathbf{1}[f(G \setminus G') \neq f(G)]. \tag{16}$$

A value of 1 means the prediction always changes, suggesting the explanation captured all the decisive structures.

Sparsity measures the compactness of the explanation relative to the original graph. For each graph $G = (V, E)$ with explanation $G' = (V', E')$,

$$Sparsity = 1 - \frac{|E'|}{|E|}. \tag{17}$$

Higher values indicate more compact explanatory subgraphs.

ER (edge ratio) is a domain-specific measure that evaluates the proportion of attack-related flows included in the explanation. If $E_{attack} \subseteq E$ denotes the set of attack-labeled edges (packet flows),

$$ER = \frac{|E' \cap E_{attack}|}{|E'|}. \tag{18}$$

It captures whether the generated explanation reflects the characteristics of the complete input graph, i.e., whether the proportion of attack-related edges in the explanation approximates or exceeds their proportion in the original traffic.

Together, these metrics ensure that the generated explanations are not only faithful to the model's predictions but also compact and semantically aligned with domain knowledge. In practice, however, they represent competing objectives, and the goal is to identify a reasonable trade-off among them (e.g., higher sparsity means more compact explanations but may also increase the risk of omitting critical information).

6.4 Environment

All experiments were conducted on Ubuntu 22.04.1 LTS running under WSL on a Windows 11 host machine. The hardware configuration included an NVIDIA RTX 3050 Ti GPU and 32 GB of system memory. The implementation was done in Python 3.12.11, using the following key libraries: PyTorch 2.4.0 with CUDA 12.4 for deep learning, DGL 2.4.0 for graph modeling, and scikit-learn 1.6.1 for data preprocessing and evaluation. Both our proposed approach and the competing baseline models were implemented within this software stack to ensure consistency and comparability.

6.5 Performance analysis (RQ1)

To evaluate model performance, we implemented three baselines, tested them on both binary and multiclass classification, and compared their results with our proposed method.

The first baseline model was a *multi-layer perceptron* (MLP), as prior studies indicate that lightweight, feature-based models can remain competitive baselines for cyber threat detection.[52] An important characteristic of this model is that it cannot directly process graph-structured data or sequences. Therefore, within each time window, we aggregated the observed network traffic into a single feature vector to be used as the model input. Specifically, we applied statistical pooling across the dataset: for each edge-level feature, we computed the mean, standard deviation, minimum, and maximum values. This yielded a single vector of size $4d$, where d denotes the dimensionality of the packet flow feature vectors. The MLP architecture consisted of two hidden layers with 256 and 128 neurons, respectively. Training was performed with a batch size of 128 and was terminated if no improvement was observed for five consecutive epochs.

The second baseline was a *Long Short-Term Memory* (LSTM). Unlike the MLP, this model can handle sequential data; therefore, the network traffic observed within each time window was ordered by start time and fed into the model as a sequence. The last hidden state was then passed through a dense layer to map it to the dimensionality corresponding to the number of classes. The LSTM employed a hidden dimension of 64. During training, we used a batch size of 128 and applied early stopping with a patience of five epochs.

The third baseline was the state-of-the-art *E-GraphSAGE* model, adapted for time-windowed operation and graph-level classification. This method operates directly on graph-structured data. Following the recommendations detailed in,[7] the model was configured with two GNN layers and ReLU activation functions. Training was performed with a batch size of 128, and early stopping with a patience of five epochs was applied.

For our proposed method, we employed three GNN layers with a hidden dimension of 128 and eight attention heads. Training was conducted similarly with a batch size of 128, a learning rate of 0.001, and early stopping with a patience of 5 epochs.

Since the proposed method is intended for online operation, computational efficiency should also be considered alongside classification performance. To this end, we provide a theoretical overview of the computational characteristics of the evaluated models with respect to the amount of traffic observed within a time window. Let e denote the number of packet flows in a time window. As each flow is mapped to one edge, e also denotes the number of graph edges. Let n denote the number of nodes, corresponding to the distinct hosts, and let d denote the dimensionality of the packet flow feature vectors.

Table 2 shows that graph construction and MLP preprocessing scale linearly with the number of packet flows in a time window, since each flow is represented as one graph edge and the feature dimensionality is fixed for a given dataset. The MLP inference cost is independent of the number of flows after aggregation, whereas the LSTM cost increases with the sequence length. The graph-based models, E-GraphSAGE and EdgeGAT, scale linearly with the size of the constructed

Table 2. Computational complexity with respect to input size.

Process	Input representation	Complexity
Graph construction	packet flows as graph edges	$O(e)$
MLP preprocessing	statistical pooling over flows	$O(e)$
MLP inference	fixed $4d$-dimensional vector	independent of e
LSTM inference	time-ordered flow sequence	linear in e
E-GraphSAGE inference	traffic graph with edge features	linear in $n + e$ per layer
EdgeGAT inference	traffic graph with edge features	linear in $n + e$ per layer, with attention over edges

Table 3. Performance of binary classification.

Dataset	Method	Precision	Recall	F1
D1	Our method	**0.9014**	**0.8861**	**0.8810**
	E-GraphSAGE	0.8974	0.8806	0.8749
	LSTM	0.8969	0.8776	0.8713
	MLP	0.8950	0.8749	0.8682
D2	Our method	**0.8740**	**0.8555**	**0.8582**
	E-GraphSAGE	0.8533	0.7780	0.7820
	LSTM	0.8483	0.7748	0.7790
	MLP	0.8189	0.7880	0.7856

graph, i.e., with the number of nodes and edges per layer. Although the proposed EdgeGAT introduces edge-level attention, it is computed only along existing graph edges rather than over all node pairs. Therefore, for a fixed architecture, the asymptotic scaling with respect to the input graph size remains linear in $n + e$ per layer. This indicates that the proposed method preserves graph-structured traffic information while maintaining a computationally feasible structure for online IDS operation, which is further supported by the empirical latency results in the following section.

The objectives of our experiments on classification performance were threefold. First, we aimed to validate that packet flow-level granularity is essential for accurate intrusion detection, and that utilizing coarser-grained representations (statistical pooling) cannot achieve equal or superior performance. Second, we sought to demonstrate that, in the context of network traffic, the structural properties of communication are generally more informative than the sequential order of flows. Finally, our goal was to establish that the proposed method can achieve state-of-the-art performance. Based on these objectives, we formulated four hypotheses:

H1. MLP is expected to underperform compared to the other models, as it relies on coarsely aggregated data that discards both structural and temporal relationships.

H2. LSTM is expected to achieve higher performance than the MLP, since it operates on packet-flow-level data. However, it is hypothesized to be less effective than graph-based methods, because sequential order provides less discriminative information than the structural properties of communication.

H3. E-GraphSAGE is expected to outperform MLP and LSTM by leveraging both packet flow-level information and graph topology in the decision process.

H4. Our proposed method is expected to achieve performance comparable to or significantly higher than that of E-GraphSAGE. This is attributed to its edge-aware attention mechanism, which enables capturing higher-order relations beyond fixed neighborhoods for intrusion detection.

For the binary classification experiments, we relabelled all attack types under a single "Attack" class, while preserving the "Normal" class for benign traffic. The results are summarized in Table 3.

On D1, the performance differences among the evaluated models are relatively small: absolute differences range between 0.4% and 0.64% in terms of precision, and between 0.61% and 1.28% in terms of F1-score. In contrast, larger discrepancies can be observed on dataset D2, where precision differences span from 2.07% to 5.51%, while F1-score differences range from 7.26% to 7.92%. An additional observation is that, although the MLP generally underperforms compared to the LSTM in most metrics, on dataset D1 their performance is nearly indistinguishable. On dataset D2, however, the MLP achieves higher recall at the expense of precision, resulting in a slightly more balanced F1-score than the LSTM. This highlights a precision–recall trade-off: while the MLP sacrifices precision, it achieves higher recall, meaning that fewer attacks remain undetected. In scenarios where maximizing sensitivity to attacks is critical - even at the cost of more false alarms - the computationally less complex MLP may be preferable over the LSTM. Importantly, its simplicity and lower computational cost make the MLP a viable candidate for deployment in resource-constrained environments,

Table 4. Performance of multiclass classification.

Dataset	Method	Precision	Recall	F1
D1	Our method	**0.9627**	0.8734	**0.9047**
	E-GraphSAGE	0.9515	**0.8896**	0.8894
	LSTM	0.9030	0.8595	0.8721
	MLP	0.9371	0.8557	0.8796
D2	Our method	**0.8912**	**0.8243**	**0.8370**
	E-GraphSAGE	0.8759	0.7638	0.7838
	LSTM	0.5711	0.7169	0.6255
	MLP	0.7963	0.6677	0.7028

such as IoT edge devices, embedded systems, and network middleboxes where computational overhead must be minimized. Finally, in both datasets, our proposed method consistently outperforms the baselines. Compared to the adapted E-GraphSAGE, our method yields a relative improvement in F1-score of 0.7% on D1 and 9.74% on D2.

The results of the multiclass classification experiments are reported in Table 4. In this setting, the performance differences among the evaluated models are more pronounced than in the binary task, with dataset D2 again exhibiting greater variability than D1.

On D1, the MLP and LSTM models achieve comparable performance, whereas on D2 the MLP substantially outperforms the LSTM in both precision and F1-score. This finding suggests, on the one hand, that the sequential ordering of packet flows plays a less critical role in classification accuracy compared to their numerical characteristics, and on the other hand, it reinforces the observation made in the binary experiments that the computationally lighter MLP model may represent a practical alternative to the more complex LSTM in certain scenarios. The results further demonstrate that graph-based methods clearly surpass both the MLP and LSTM baselines, indicating that the structural properties of communication contain crucial information for detecting network anomalies. Across both datasets, our proposed method consistently outperforms the adapted E-GraphSAGE model in most metrics, with relative improvements in F1-score of 1.72% on D1 and 6.79% on D2, however, that on D1 the E-GraphSAGE achieved higher recall.

Since datasets are relatively imbalanced, as shown in Figure 4, relying solely on aggregate metrics such as precision, recall, and F1-score may obscure important differences across classes. In particular, it is essential to ensure that the strong overall performance of our method is not merely the result of high accuracy on the majority classes but also reflects robustness across minority attack categories. For this reason, we further examine the confusion matrices, which provide a more detailed view of class-level behavior and allow us to assess whether the proposed model achieves consistent performance across different types of network traffic.

The normalized confusion matrices for both datasets are reported in Table 5. Darker shading indicates values closer to 1, while lighter shading indicates values closer to 0. For reference, the last column shows the number of samples in each ground-truth class. Based on results from both datasets, our method demonstrates nearly balanced performance across all labels, though moderate deviations are observed for a few classes.

For dataset D1, the misclassification of MITM attacks is most likely due to the small number of available samples, as this attack type accounts for only 0.005% of the dataset. Moreover, it is evident that the model erroneously classified 22% of normal traffic instances as MITM, suggesting that the scarcity of training samples for this class not only leads to poor recognition of the attack itself but also results in assigning unrelated samples to the MITM category. Taken together, these findings indicate that in real-world deployments, practitioners should carefully consider whether to include extremely rare attack types during model training. Doing so may not only increase the risk of misclassifying such attacks but may also result in false positives or mislabeled alerts originating from other classes. Indeed, some prior works (e.g.,[14]) excluded the MITM class from both training and evaluation. Furthermore, the inherently stealthy nature of MITM attacks, which are often designed to remain undetected for extended periods, may further explain why this class is particularly challenging for the model to identify. Notably, while the model correctly identified 62% of MITM samples, this was accompanied by a tendency to over-predict the MITM class, leading to a share of other samples being labeled as MITM. At the same time, the model achieved remarkably robust performance on other minority classes, such as ransomware and scanning, with recalls of 84% and 85%, respectively. This suggests that the observed difficulties are not solely due to class imbalance, but rather specific to the characteristics of the MITM class itself.

In the case of D2, the confusion matrix also exhibits a strong diagonal, indicating that the model robustly classified most of the majority and minority classes. The most notable deviation appears in the DoS class, where 44% of the samples were misclassified as normal. This is primarily due to the limited number of available samples. It is also worth noting that DoS attacks may not initially cause substantial disruption to system operations and are often easier to mitigate in practice,

Table 5. Normalized confusion matrices for D1 and D2.

D1		Predicted label										Support
		Backd.	DDoS	DoS	Injection	MITM	Normal	Passwd.	Ransomw.	Scan	XSS	
Ground truth	Backd.	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	933
	DDoS	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	602
	DoS	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	311
	Injection	0.01	0.00	0.00	0.94	0.01	0.02	0.01	0.00	0.01	0.00	148
	MITM	0.00	0.00	0.00	0.06	0.62	0.12	0.03	0.00	0.15	0.03	34
	Normal	0.00	0.00	0.00	0.05	0.22	0.69	0.00	0.03	0.00	0.00	1,778
	Passwd.	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	505
	Ransomw.	0.00	0.00	0.00	0.02	0.04	0.07	0.00	0.84	0.00	0.02	45
	Scan	0.00	0.00	0.00	0.03	0.07	0.05	0.01	0.00	0.85	0.00	142
	XSS	0.00	0.00	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.98	250

D2		Predicted label							Support
		Bot	Brutef.	DDoS	DoS	Infiltr.	Web	Normal	
Ground truth	Bot	0.99	0.00	0.00	0.00	0.01	0.00	0.00	203
	Brutef.	0.00	1.00	0.00	0.00	0.00	0.00	0.00	114
	DDoS	0.00	0.00	1.00	0.00	0.00	0.00	0.00	94
	DoS	0.00	0.00	0.00	0.56	0.00	0.00	0.44	75
	Infiltr.	0.00	0.00	0.00	0.00	0.98	0.00	0.02	638
	Web	0.00	0.00	0.00	0.00	0.00	0.83	0.16	225
	Normal	0.00	0.00	0.00	0.00	0.24	0.00	0.76	2,470

as they typically originate from fewer hosts than DDoS attacks. By contrast, the model recognized DDoS traffic with 100% accuracy, underscoring its ability to capture the characteristic patterns of large-scale distributed attacks. For the web-based attacks, the model achieved a recall of 83%. This indicates that the traffic patterns of these attacks partly resemble those of benign web traffic, making this class more challenging to cleanly separate from normal activity. Regarding the normal class, the model correctly identified 76% of the samples, while 24% were incorrectly labeled as infiltration. This suggests that infiltration attacks are particularly difficult to detect, as their traffic patterns are deliberately designed to resemble benign activity. The resulting misclassifications were concentrated almost entirely in the normal–infiltration class pair, highlighting a specific boundary challenge rather than a general weakness in recognizing benign traffic.

The experimental results provide mixed support for our hypotheses. H1 is broadly confirmed, as the MLP generally underperformed compared to graph-based methods, though it slightly outperformed the LSTM on D2 due to a precision–recall trade-off. H2 is only partially supported: while the LSTM surpassed the MLP on D1, this advantage did not hold on D2, suggesting limited benefit from sequential ordering. H3 is confirmed, with E-GraphSAGE consistently outperforming both MLP and LSTM, demonstrating that structural properties of communication graphs provide highly informative signals for intrusion detection. H4 is confirmed, as our proposed method at least matched and, in most cases, significantly exceeded E-GraphSAGE across both tasks.

Answer to RQ1:
The performance analysis indicates that the proposed method consistently outperforms the state-of-the-art E-GraphSAGE approach across both binary and multiclass classification tasks. In the binary setting, we observed relative improvements of 0.7% on D1 and 9.74% on D2. In the multiclass setting, the gains were 1.72% on D1 and 6.79% on D2. Furthermore, the analysis of the confusion matrices indicates that the model's performance is robust even for minority classes, suggesting that the improvements also extend to less-represented attack types. Therefore, the model is considered an effective approach for intrusion detection across diverse datasets and attack categories.

Table 6. Average graph construction, inference, and overall latency across different time window sizes.

Dataset	Time window (s)	Graph construction time (ms)	Inference time (ms)	Overall latency (ms)
D1	5	17.501	6.236	23.737
	10	34.555	5.658	40.213
	20	75.762	6.590	82.352
	40	138.341	7.536	145.877
D2	5	30.055	6.388	36.443
	10	64.840	6.041	70.881
	20	146.446	6.334	152.780
	40	240.705	7.111	247.816

6.6 Time window sensitivity analysis (RQ2)

To investigate the sensitivity of the model to the time window size, the evaluated datasets were segmented into 5, 10, 20, and 40-s windows. Experiments were then conducted for each window size individually to examine the resulting precision, recall, and F1-score values, as well as the latency of the detection pipeline.

Table 6 presents the latency-related results for both evaluated datasets, the average processing time of the examined time windows are reported. Graph construction time is the average time required to build the traffic graph from packet flows observed during the given time window. The reported times represent a practical upper bound, since traffic graph construction can be performed within the time window immediately upon monitoring the corresponding flows, rather than only after the window has fully elapsed. Inference time indicates the time required by the decision model to produce the output label from the input graph. Overall latency corresponds to the sum of these two processing times.

Graph construction times increase approximately in proportion to the time window size across both datasets, since larger windows contain more packet flows, resulting in larger traffic graphs with more nodes and edges to process. Furthermore, D2 consistently exhibits higher graph construction times than D1 across all evaluated window sizes. This can be attributed to the greater complexity of D2, which contains more raw features and more heterogeneous communication patterns involving more hosts. Consequently, the resulting graph representations require more extensive preprocessing and graph construction operations, however graph construction remained below 250 ms even for the largest evaluated time window.

In contrast, the inference time remains nearly constant across all evaluated window sizes and datasets, with only minor variations. This indicates that the computational cost of the forward pass is stable and robust to settings of the temporal segmentation.

Figure 5 shows the effect of time window size on classification performance. Across both datasets, precision remains relatively stable as the window size increases. This suggests that the reliability of the model's predicted class labels is weakly affected by the selected temporal window size. In contrast, recall increases consistently with larger time windows, with a moderate improvement on D1 and a more pronounced improvement on D2. This indicates that larger windows provide richer temporal and structural context, enabling the model to assign a higher proportion of true instances to their correct classes. Consequently, the F1-score also improves with increasing window size on both datasets. These results suggest that the main benefit of larger time windows is an increased ability to correctly classify a larger fraction of true class instances.

Answer to RQ2:
Size of the time windows has a clear impact on detection performance and computational efficiency. Graph construction time increased approximately proportionally with the window size, while inference time remained stable; nevertheless, the overall latency remained below 250 ms in the worst case. Larger windows mainly improved recall and F1-score, while precision remained nearly unchanged, indicating that wider temporal context helps the model identify a larger fraction of true attack instances. In real-world deployment, larger windows can introduce additional detection delay, making window size a practical trade-off that may be mitigated through sliding-window operation.

6.7 Hyperparameter sensitivity analysis (RQ3)

The analysis of hyperparameter sensitivity serves multiple purposes. First, we aimed to assess whether variations in individual hyperparameters substantially affect the model's performance. Second, by examining performance trends across two diverse datasets, we sought to identify generally optimal values to inform real-world implementations. Beyond these objectives, the analysis also highlights which hyperparameters should be prioritized during tuning versus those with limited impact on performance.

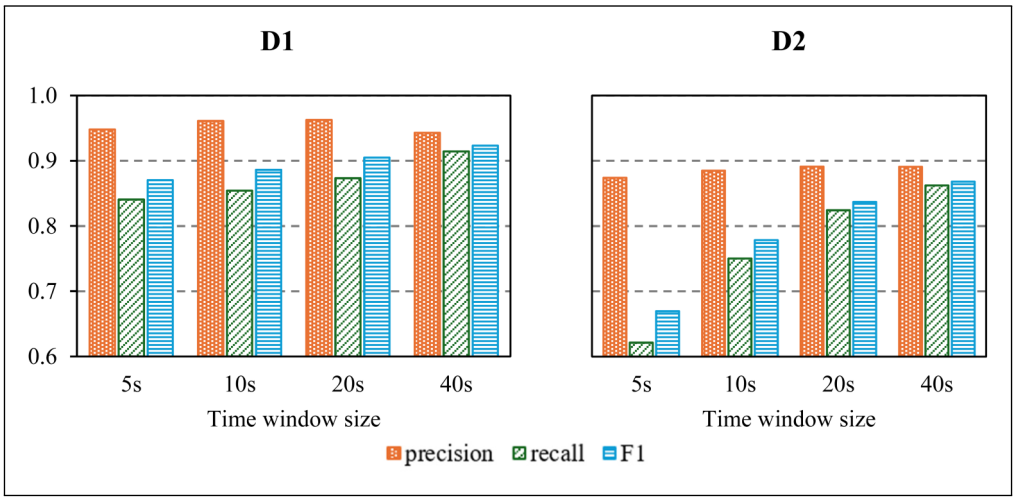


Figure 5. Effect of time window size on precision, recall, and F1-score.

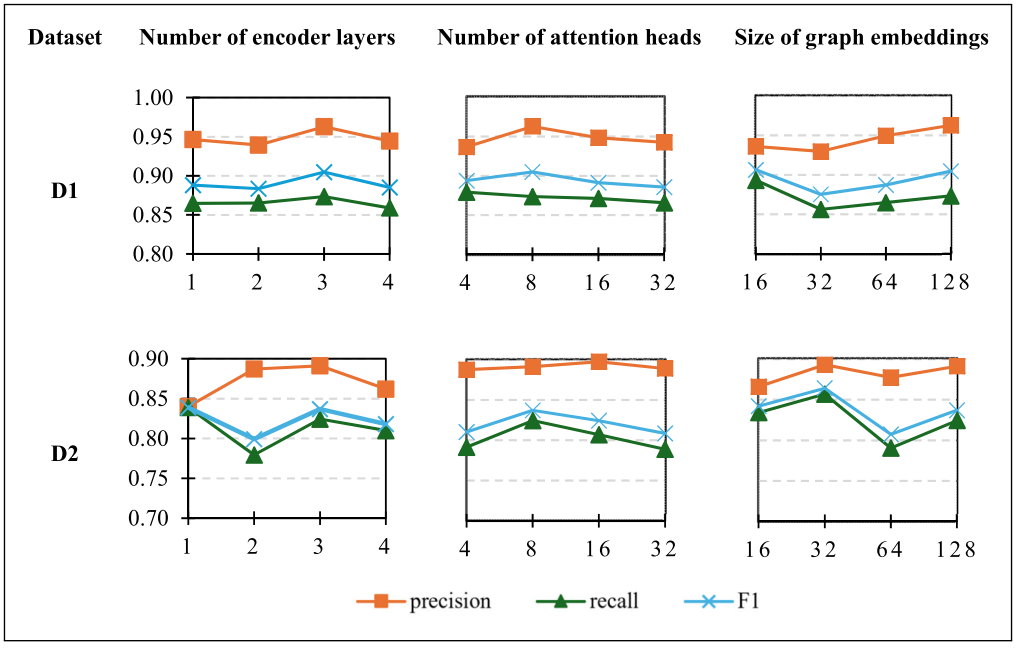


Figure 6. Effect of hyperparameters on the model's performance.

Our analysis focused on three key hyperparameters: the number of graph encoder layers, the number of attention heads, and the dimensionality of the graph embeddings. For the encoder layers, deeper architectures can capture more complex dependencies in the input data. However, each additional layer increases computational cost linearly, and excessive depth may reduce time efficiency without substantial performance gains. The number of attention heads determines how many relational patterns the model can capture in parallel. Too few heads may limit expressiveness and reduce performance, whereas too many heads often fail to uncover additional dependencies and primarily increase computational overhead. Finally, the graph embedding size directly impacts the model's representational capacity. Smaller embeddings may be insufficient to distinguish heterogeneous traffic patterns, while excessively large embeddings can lead to overfitting and poor generalization.

The results of our experiments are illustrated in Figure 6. Regarding the number of encoder layers, the model achieved its best performance on D1 with three layers, although the absolute difference between the lowest and highest F1-scores was only 2.1%. In contrast, D2 exhibited greater variation: using three layers yielded a 6.2% relative improvement in precision over a single layer, while recall remained largely unchanged. Based on these findings, we consider three encoder layers to be an optimal choice.

With respect to the number of attention heads, inspection of both D1 and D2 curves indicates that using 8 heads yields the best overall performance. On D1, the gap between the lowest and highest F1-scores was only 1.1%, while on dataset D2 the corresponding gap was 2.9%. These results suggest that 8 attention heads represent a generally optimal configuration.

With respect to embedding dimensionality, the results show that increasing the hidden size does not uniformly translate into better performance. On D1, performance steadily improved with larger embeddings, reaching the highest F1-score at 128 dimensions. In contrast, on D2, the best performance was achieved with 32 dimensions, while further increasing the embedding size led to a slight drop in accuracy. These results indicate that the optimal embedding size is dataset-dependent: while larger dimensions can capture richer structural information, they may also introduce redundancy or noise that reduces generalization. While D1 clearly benefited from the larger embedding size and D2 achieved its peak performance at 32 dimensions, the 128-dimensional setting provides consistently competitive results and can serve as a robust default choice in practice.

Answer to RQ3:
The analysis confirms that the examined hyperparameters have a substantial impact on model performance. For practical implementations, either during prototyping or as a starting point for hyperparameter optimization, we recommend using 3 encoder layers, 8 attention heads, and an embedding dimensionality of 128. While the optimal values for the number of encoder layers and attention heads were consistent across both datasets, the choice of embedding size proved more sensitive to dataset-specific characteristics.

6.8 Explainability analysis (RQ4)

The objective of the explainability analysis was to assess how accurately the generated subgraph reflects the properties of the complete input graphs upon which the model based its predictive decision. In this context, we conducted an exploratory qualitative investigation, complemented by a quantitative analysis utilizing non–domain-specific and domain-specific metrics.

To examine the qualitative characteristics, we visualized in Figure 7 the explanatory subgraphs corresponding to the input graphs shown in Figure 1, constructed from the top 10th percentile of edges by importance ($p = 0.1$). For reference, the explanation of normal traffic prediction is also depicted, although in practice generating such explanations is not necessarily required; it is included here for visual symmetry with Figure 1. For each attack type, the explanation preserves the key structural motifs previously identified in the input graphs, indicating that these motifs are indeed informative and play a decisive role in the model's predictions. Across all attack categories, the most characteristic connections (e.g., central hubs in DDoS and backdoor attacks, intermediary relays in MITM, or dense clustered structures in XSS and injection attacks) remain visible, suggesting that the model consistently retains informative edges while producing compact explanations.

In the quantitative analysis, we evaluated non–domain-specific characteristics by measuring the faithfulness of the explanations to the model's original predictions, utilizing the $fidelity_+$ and $fidelity_-$ metrics. As a domain-specific characteristic, we also assessed the proportion of edges in the explanations that correspond to packet flows associated with malicious intent (ER = edge ratio). These metrics were evaluated as a function of *sparsity*, which reflects the compactness of the explanation, thereby highlighting the inherent trade-offs between compactness and accuracy.

Figure 8 presents the measured values on D1 and D2 for p varying between 0 and 1. In the case of D1, we observed that even a highly compact subset of the original graph is sufficient for a faithful explanation: fidelity_- decreases only to 0.727 at 83% sparsity. The fidelity_+ varies nearly linearly with p, suggesting that the model's decisions rely on a broad set of moderately important edges, rather than only on a few dominant ones. The attack edge ratio (ER) remains nearly constant across different p values, varying between 93% and 94%, indicating that even the most compact explanations accurately represent characteristics of the original network traffic.

In the case of D2, fidelity_- increases with p: even at 29.3% sparsity it remains almost perfect (0.961), and at 59.7% sparsity it still shows a fidelity level of 0.237. A temporary drop in fidelity_+ between $p = 0.2$ and 0.3 suggests a transition phase in which the explanation starts to include less decisive edges alongside the most critical ones, temporarily weakening the impact of their removal on the model's decision. The attack edge ratio (ER) indicates that the explanations effectively capture the behavior of the input data: while the proportion of attack-related edges in the original graphs is 16.8%, it rises to 21.7% at 59.7% sparsity, showing that the model focuses more on attack-related connections during its decision-making process. At higher sparsity levels, however, the edge ratio gradually decreases, reflecting that additional edges incorporated into the explanation include more benign communication flows.

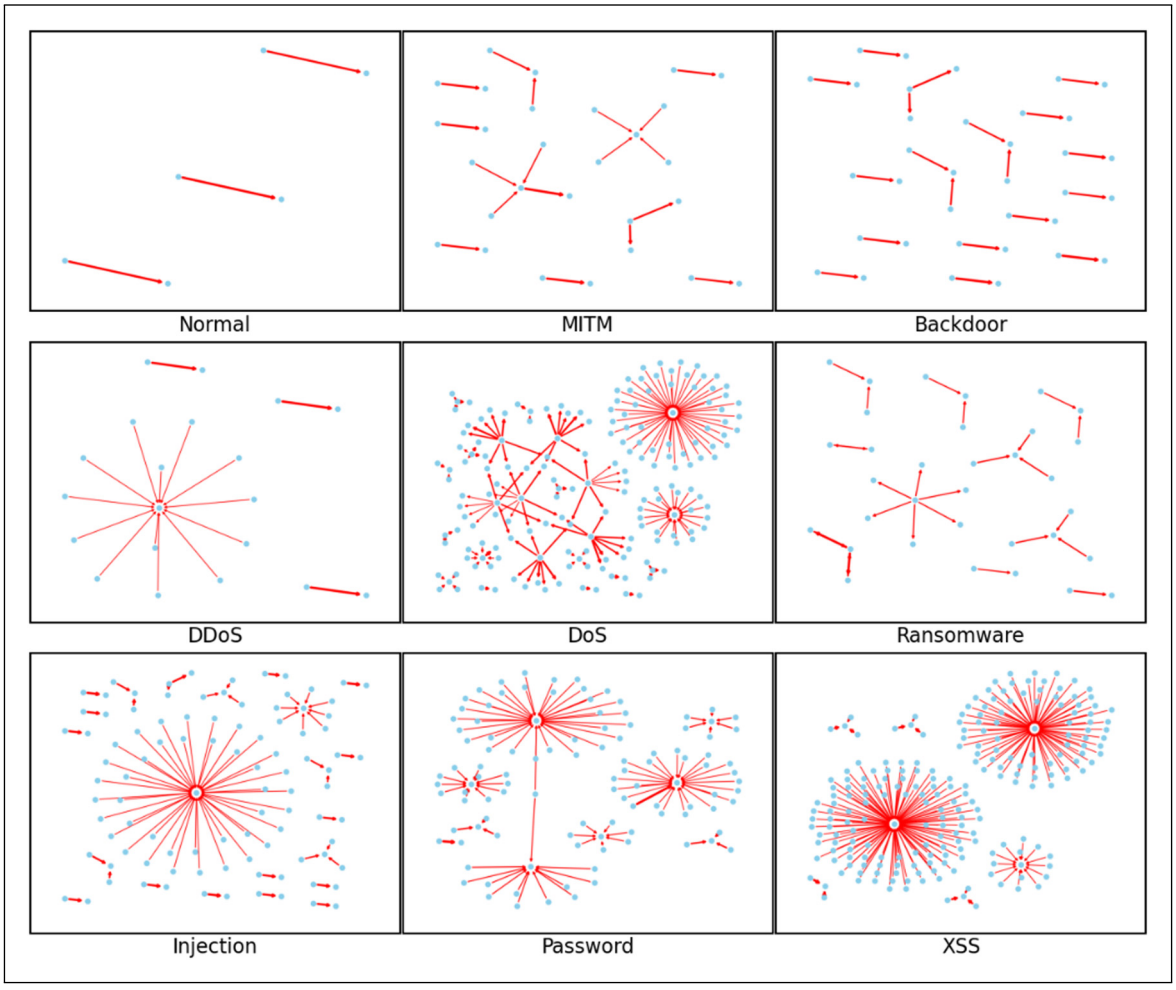


Figure 7. Explanatory subgraphs for normal operation and different attacks in D1 (p=0.1).

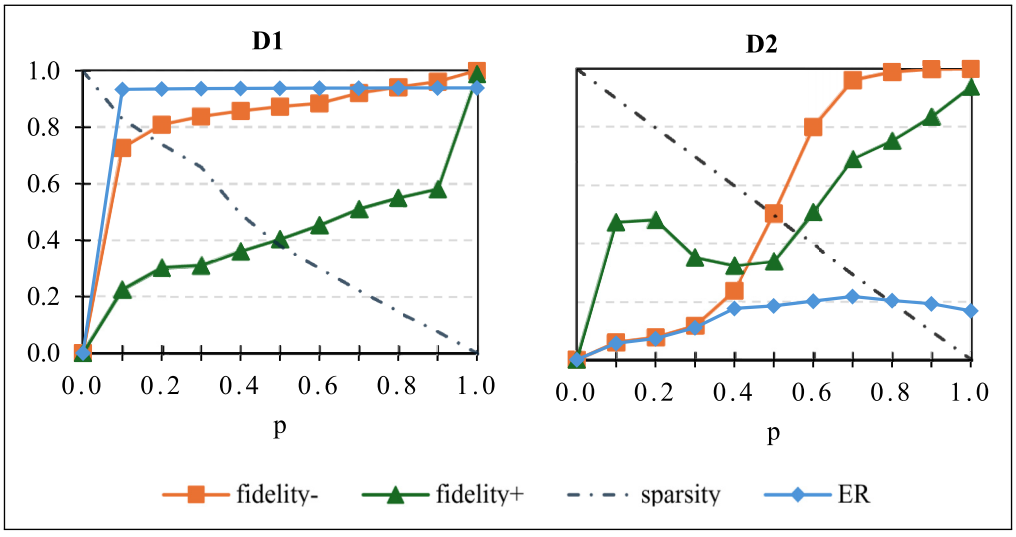


Figure 8. Explainability metrics as a function of p.

Answer to RQ4:
The explainability analysis shows that the model produces explanations that are both faithful and semantically meaningful. The explanatory subgraphs retain the key structural motifs of the input graphs across attack types, and the quantitative results confirm acceptable fidelity and stable attack edge ratios across diverse datasets.

7 Case study

To demonstrate the operation of the proposed intrusion detection framework under realistic conditions, we conducted a case study using D2. We focused on a scenario in which the system initially operates normally and is subsequently subjected to a brute-force attack targeting the SSH service accessible at *172.31.69.25:22*, originating from an attacker at *13.58.98.64*. This attack type exhibits a distinct topological pattern characterized by a highly centralized communication structure, in which the attacker attempts to gain access by repeatedly sending login requests from multiple ports toward the SSH server while iterating through a predefined password dictionary.

Figure 9 shows the network traffic graphs captured at the onset of the attack, with the centers of the five largest components labeled. During the first 20-s interval, the system operates normally. In addition to the DNS server running on port 53, the most notable activity involves remote desktop connections on port 3389, which stand out against the otherwise predominantly point-to-point communication pattern. In the second interval, a new cluster emerges in the traffic, exhibiting a star-shaped topology centered on the SSH service hosted at *172.31.69.25*. The model correctly classifies this pattern as a brute-force attack with 99.67% confidence. As the volume of traffic directed toward the SSH service increases over time, the model reports predictions with progressively higher confidence.

Figure 10 (left) shows the explanation generated for the prediction corresponding to the second interval, which marks the beginning of the attack. The explanation graph was produced using mean aggregation across attention heads and max aggregation across encoder layers, with a sparsity parameter of $p = 0.6$. The visualization reveals that, while the model did not entirely disregard benign point-to-point communications, it primarily focused on larger clusters within the network traffic. Among these, the largest cluster corresponds to the attacked SSH service, indicating that the model assigned significant attention to flows associated with 172.31.69.25:22. Based on this explanation, system administrators can further inspect the packet flows related to this service, as illustrated in Figure 10 (right), and possibly examine the corresponding SSH daemon logs, which reveal numerous failed login attempts. Such information supports the initiation of appropriate mitigation measures, e.g., enabling whitelists, introducing throttling, or enforcing intentional latency in the SSH login process.

The case study demonstrates that the proposed GNN-based intrusion detection framework can accurately and rapidly identify attacks while providing interpretable insights into their structural characteristics. The attention-based explanation highlights the key communication flows involved in the intrusion, supporting timely understanding and mitigation. Overall, the results confirm that the framework effectively combines accuracy, low detection latency, and explainability in practical network security scenarios.

8 Limitations

While the proposed model demonstrated strong performance across diverse datasets in both binary and multiclass classification tasks, several aspects warrant further investigation and potential improvement.

The current version of the model is designed to detect only a single attack type at a time window. This limitation could be addressed by employing multi-label classification strategies (e.g., training multiple binary classifiers or applying adaptive thresholding mechanisms) that allow overlapping detections of concurrent attack scenarios. Also, while the temporal train-test split provides a realistic chronological evaluation, concept drift was not explicitly quantified and remains an important direction for further investigation.

The model is trained to recognize only previously observed attack types and therefore cannot identify novel ones. Future work could explore incremental and adversarial learning strategies to enable continuous adaptation and improved resilience against novel attacks.

The proposed explanation method relies solely on attention scores, which may not always capture causal relationships. This could be extended by integrating model-agnostic explanation frameworks such as SHAP,[53] its graph-aligned variants[54] or Integrated Gradients,[55] allowing the transition from attention-based explanations to more robust causal interpretations. Such extensions could also support the development of graph-based root cause localization methods.

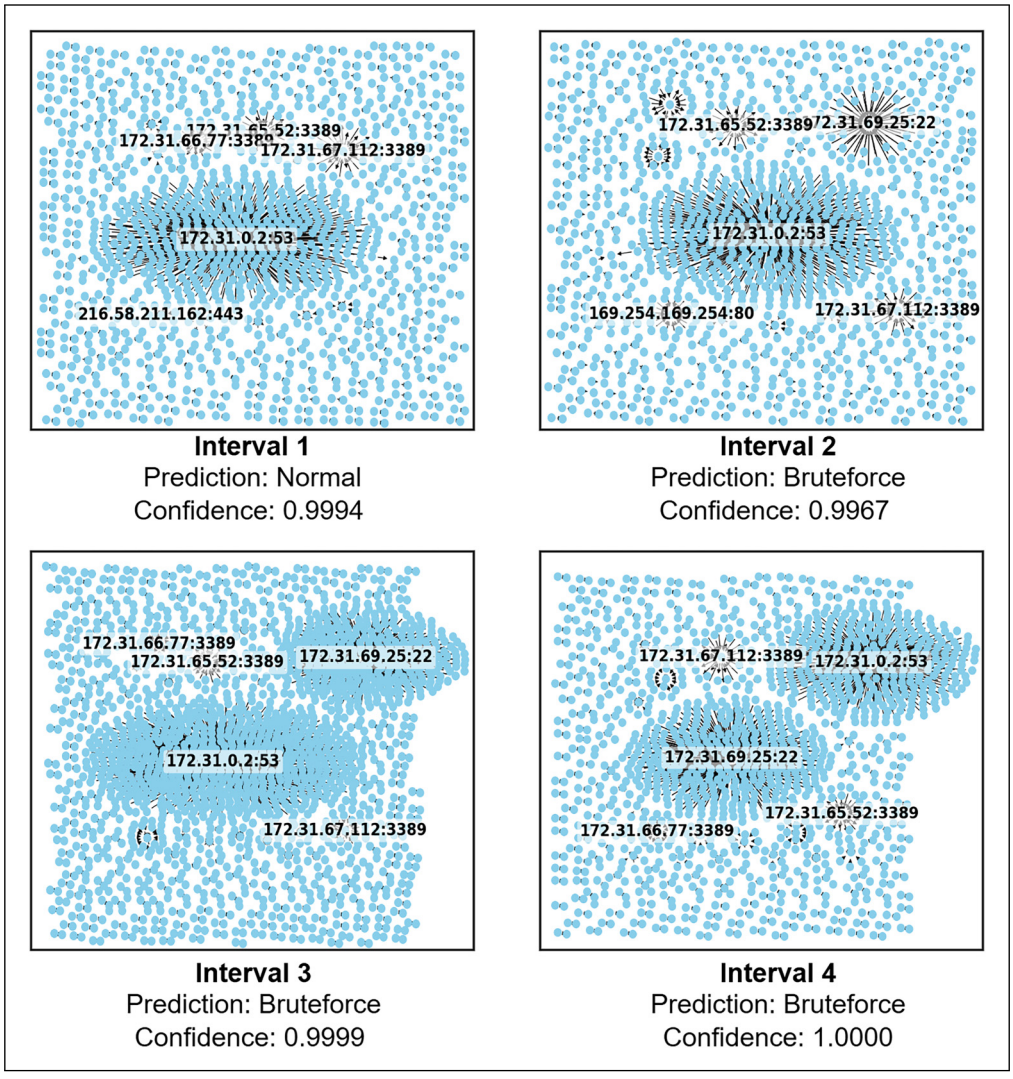


Figure 9. Network traffic graphs captured during the brute-force attack targeting the SSH service, where each graph represents the communication observed within a consecutive 20-s time window.

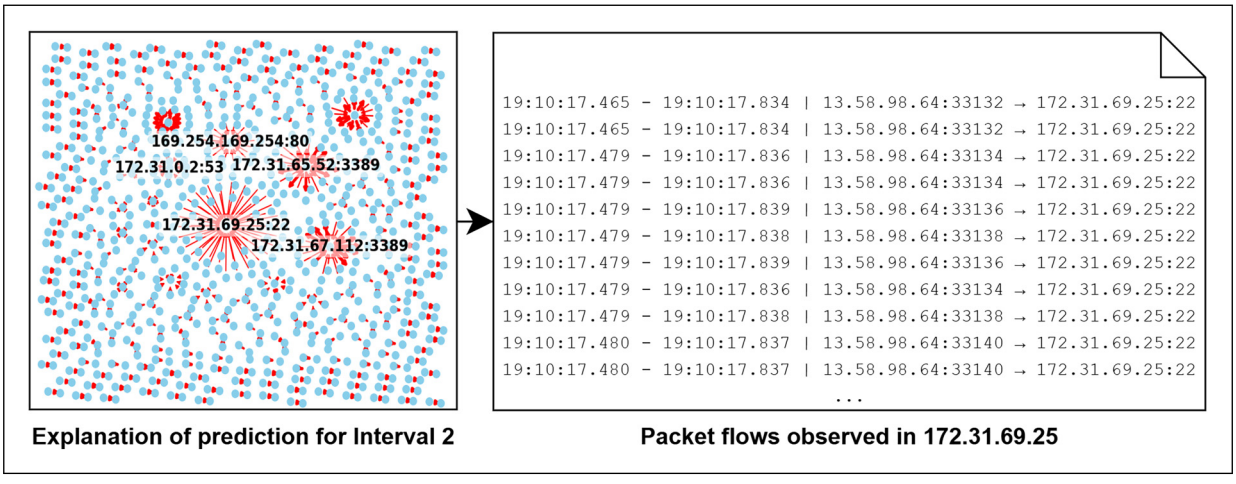


Figure 10. Explanation graph and monitored packet flows of the attacked host.

Scalability may become a concern when processing extremely large input graphs (e.g., under very heavy DDoS or scanning attacks). In such cases, sampling strategies like random undersampling, neighborhood sampling, or adaptive subgraph extraction could be investigated to maintain computational efficiency without significantly compromising detection accuracy.

9 Conclusion and future work

This study presented a graph neural network-based intrusion detection method designed for online operation, capable of detecting anomalies in system behavior based on temporally segmented network traffic. By introducing a time-window-based graph construction strategy, the proposed approach enables early detection of security incidents through periodic classification of the system's state. The results demonstrated that graph-based modelling effectively captures complex communication structures and provides significant advantages over aggregated or sequence-based methods. The adopted edge-aware attentive GNN architecture, combined with the proposed time-windowed graph construction and classification pipeline, achieved state-of-the-art performance across diverse datasets, confirming its suitability for online, time-constrained intrusion detection tasks. Furthermore, the model can generate explanation graphs that highlight the most relevant communication flows behind the prediction, thereby supporting human operators in understanding attack patterns and implementing timely mitigation measures. Overall, the proposed framework combines accurate detection with explainability, offering a practical decision-support tool for real-world cybersecurity applications.

Future research will focus on extending the explainability mechanism into an interpretation framework by incorporating domain knowledge and causal reasoning into the analysis of model decisions. Such an enhancement would enable the identification of underlying relationships between network entities, improving both the transparency and diagnostic value of the model's outputs. In addition, the proposed IDS method could be extended toward an IPS by enabling automatic remediation strategies based on detected malicious flows. An intermediate step in this direction would be to incorporate edge-level classification, allowing the fine-grained localization of malicious communications. Building on this capability, automated countermeasures, such as isolating compromised nodes or applying targeted defence rules, could then transform the system from a proactive detection framework into an adaptive, self-protecting cybersecurity solution.

ORCID iDs

Áron Kiss https://orcid.org/0000-0003-4549-4107
Károly Nehéz https://orcid.org/0000-0002-6953-3898
Olivér Hornyák https://orcid.org/0000-0003-0989-6109

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was supported by the University of Miskolc.

Declaration of conflicting interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Data and source code

The used datasets are available at[49] and.[50]
The source code is available at https://github.com/aron123/Online-IDS-GNN.

References

1. Diana L, Dini P and Paolini D. Overview on intrusion detection systems for computers networking security. *Computers* 2025; 14: 87. doi: 10.3390/computers14030087.
2. Prakash R, Jyoti N and Manjunatha S. A survey of security challenges, attacks in IoT. *E3S Web Conf* 2024; 491: 04018. doi: 10.1051/e3sconf/202449104018.
3. Awaad AM, Alheeti KMA and Najem AKAH. Anomaly-Based IDS (intrusion detection system) for cyber-physical systems. *Mesop J Big Data* 2024; 2024: 230–240. doi: 10.58496/MJBD/2024/017.
4. Vasilică B-V, Anton F-D and Ioniță AD. Evaluation of IBM watson services for IDS implementation. *UPB Sci Bull Ser C* 2024; 86: 183–198, [Online]. Available: https://www.scientificbulletin.upb.ro/rev_docs_arhiva/rez265_278516.pdf
5. Zhong M, Lin M, Zhang C, et al. A survey on graph neural networks for intrusion detection systems: methods, trends and challenges. *Comput Secur* 2024; 141: 103821. doi: 10.1016/j.cose.2024.103821.

6. Bilot T, Madhoun NE, Agha KA, et al. Graph neural networks for intrusion detection: a survey. *IEEE Access* 2023; 11: 49114–49139. doi: 10.1109/ACCESS.2023.3275789.
7. Lo WW, Layeghy S, Sarhan M, et al. E-GraphSAGE: a graph neural network based intrusion detection system for IoT. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, April 2022, pp. 1–9. IEEE. doi: 10.1109/NOMS54207.2022.9789878.
8. Altaf T, Wang X, Ni W, et al. GNN-based network traffic analysis for the detection of sequential attacks in IoT. *Electronics (Basel)* 2024; 13: 2274. doi: 10.3390/electronics13122274.
9. Sun Z, Teixeira AMH and Toor S. GNN-IDS: graph neural network based intrusion detection system. In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*, July 2024, pp. 1–12. ACM. doi: 10.1145/3664476.3664515.
10. Chang L and Branco P. Embedding residuals in graph-based solutions: the E-ResSAGE and E-ResGAT algorithms. A case study in intrusion detection. *Appl Intell* 2024; 54: 6025–6040. doi: 10.1007/s10489-024-05404-2.
11. Lo WW, Kulatilleke G, Sarhan M, et al. XG-BoT: an explainable deep graph neural network for botnet detection and forensics. *Internet Things* 2023; 22: 100747. doi: 10.1016/j.iot.2023.100747.
12. Li Y, et al. GraphDDoS: effective DDoS attack detection using graph neural networks. In: *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, May 2022, pp. 1275–1280. IEEE. doi: 10.1109/CSCWD54268.2022.9776097.
13. Pujol-Perich D, Suarez-Varela J, Cabellos-Aparicio A, et al. Unveiling the potential of graph neural networks for robust intrusion detection. *ACM Sigmetrics Perform Eval Rev* 2022; 49: 111–117. doi: 10.1145/3543146.3543171.
14. Venturi A, Pellegrini D, Andreolini M, et al. Practical evaluation of graph neural networks in network intrusion detection. In: *Proceedings of ITASEC 2023: The Italian Conference on CyberSecurity, CEUR Workshop Proceedings*, 2023. [Online]. Available: https://ceur-ws.org/Vol-3488/paper29.pdf
15. Steingartner W, Galinec D and Kozina A. Threat defense: cyber deception approach and education for resilience in hybrid threats model. *Symmetry (Basel)* 2021; 13: 597. doi: 10.3390/sym13040597.
16. Satilmiş H, Akleylek S and Tok ZY. A systematic literature review on host-based intrusion detection systems. *IEEE Access* 2024; 12: 27237–27266. doi: 10.1109/ACCESS.2024.3367004.
17. Shamim A, Fayyaz B and Balakrishnan V. Layered defense in depth model for IT organizations. In: *2nd International Conference on Innovations in Engineering and Technology (ICCET'2014) Sept. 19-20, 2014 Penang (Malaysia)*, 2014, pp. 21–24. International Institute of Engineers. doi: 10.15242/IIE.E0914047.
18. Kanellopoulos D, Sharma VK, Panagiotakopoulos T, et al. Networking architectures and protocols for IoT applications in smart cities: recent developments and perspectives. *Electronics (Basel)* 2023; 12: 2490. doi: 10.3390/electronics12112490.
19. Heidari A and Jamali MAJ. Internet of things intrusion detection systems: a comprehensive review and future directions. *Cluster Comput* 2023; 26: 3753–3780. doi: 10.1007/s10586-022-03776-z.
20. Scarselli F, Gori M, Tsoi AC, et al. The graph neural network model. *IEEE Trans Neural Netw* 2009; 20: 61–80. doi: 10.1109/TNN.2008.2005605.
21. Kovács L and Jlidi A. Graphs unveiled: graph neural networks and graph generation. *Prod Syst Inf Eng* 2023; 11: 62–76. doi: 10.32968/psaie.2023.1.5
22. Ying C, et al. Do transformers really perform bad for graph representation?. 2021, *arXiv*. doi: 10.48550/ARXIV.2106.05234.
23. Liu C, et al. Graph pooling for graph neural networks: progress, challenges, and opportunities. In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, August 2023, pp. 6712–6722. International Joint Conferences on Artificial Intelligence Organization. doi: 10.24963/ijcai.2023/752.
24. Veličković P, Cucurull G, Casanova A, et al. Graph attention networks. 2018, *arXiv*. doi: 10.48550/arXiv.1710.10903.
25. Tahmid MT, Zaman TA and Rahman MS. GraFusionNet: Integrating node, edge, and semantic features for enhanced graph representations. November 2024. doi: 10.1101/2024.11.22.624875.
26. Gong L and Cheng Q. Exploiting edge features for graph neural networks. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019, pp. 9203–9211. IEEE. doi: 10.1109/CVPR.2019.00943.
27. Harary F and Norman RZ. Some properties of line digraphs. *Rend Circ Mat Palermo* 1960; 9: 161–168. doi: 10.1007/BF02854581.
28. Schlichtkrull M, Kipf TN, Bloem P, et al. Modeling relational data with graph convolutional networks. 2018: 593–607. doi: 10.1007/978-3-319-93417-4_38.
29. Wang Y, Sun Y, Liu Z, et al. Dynamic graph CNN for learning on point clouds. *ACM Trans Graph* 2019; 38: 1–12. doi: 10.1145/3326362.
30. Hu W, et al. Strategies for pre-training graph neural networks. 2020, *arXiv*. https://doi.org/10.48550/arXiv.1905.12265.
31. Monninger T, et al. SCENE: reasoning about traffic scenes using heterogeneous graph neural networks. *IEEE Robot Autom Lett* 2023; 8: 1531–1538. doi: 10.1109/LRA.2023.3234771.
32. Churcher A, et al. An experimental analysis of attack classification using machine learning in IoT networks. *Sensors* 2021; 21: 446. doi: 10.3390/s21020446.

33. Altunay HC and Albayrak Z. A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks. *Eng Sci Technol Int J* 2023; 38: 101322. doi: 10.1016/j.jestch.2022.101322.
34. Parfenov D, Grishina L, Zhigalov A, et al. Investigation of the impact effectiveness of adversarial data leakage attacks on the machine learning models. *ITM Web Conf* 2024; 59: 04011. doi: 10.1051/itmconf/20245904011.
35. Zhang B, Li J, Ward L, et al. Deep graph embedding for IoT botnet traffic detection. *Secur Commun Netw* 2023; 2023: 1–10. doi: 10.1155/2023/9796912.
36. Friji H, Olivereau A and Sarkiss M. Efficient network representation for GNN-based intrusion detection. 2023: 532–554. doi: 10.1007/978-3-031-33488-7_20.
37. Hamilton WL, Ying R and Leskovec J. Inductive representation learning on large graphs. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS '17)*, 2017, pp. 1025–1035. Curran Associates Inc.. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf
38. Hong W, et al. A graph empowered insider threat detection framework based on daily activities. *ISA Trans* 2023; 141: 84–92. doi: 10.1016/j.isatra.2023.06.030.
39. Saxena S, Grover J and Singhal S. Exploring graph neural networks for robust network intrusion detection. *Procedia Comput Sci* 2025; 258: 3630–3639. doi: 10.1016/j.procs.2025.04.618.
40. Caville E, Lo WW, Layeghy S, et al. Anomal-E: a self-supervised network intrusion detection system based on graph neural networks. *Knowl Based Syst* 2022; 258: 110030. doi: 10.1016/j.knosys.2022.110030.
41. Termos M, Ghalmane Z, Brahmia M-A, et al. Enhancing IoT network intrusion detection with a new GraphSAGE embedding algorithm using centrality measures. In: *Proceedings of the 10th International Conference on Internet of Things, Big Data and Security*, 2025, pp. 329–336. SCITEPRESS - Science and Technology Publications. doi: 10.5220/0013363000003944.
42. Deng P and Huang Y. Edge-featured multi-hop attention graph neural network for intrusion detection system. *Comput Secur* 2025; 148: 104132. doi: 10.1016/j.cose.2024.104132.
43. Van Langendonck L, Castell-Uroz I and Barlet-Ros P. PPT-GNN: a practical pretrained spatio-temporal graph neural network for network security. In: *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, June 2025, pp. 169–175. IEEE. doi: 10.1109/EuroSPW67616.2025.00026.
44. Miani RS, Bernardo GDG, Cassales GW, et al. A survey of data stream-based intrusion detection systems. *IEEE Access* 2025; 13: 72953–72983. doi: 10.1109/ACCESS.2025.3561105.
45. Zhou P. A survey of streaming data anomaly detection in network security. *PeerJ Comput Sci* 2025; 11: e3066. doi: 10.7717/peerj-cs.3066.
46. Luša R, Pintar D and Vranić M. TE-G-SAGE: explainable edge-aware graph neural networks for network intrusion detection. *Modelling* 2025; 6: 165. doi: 10.3390/modelling6040165.
47. Liu J and Guo M. DIGNN-A: real-time network intrusion detection with integrated neural networks based on dynamic graph. *CMC* 2025; 82: 817–842. doi: 10.32604/cmc.2024.057660.
48. Xiao Q, Liu J, Wang Q, et al. Towards network anomaly detection using graph embedding. 2020: 156–169. doi: 10.1007/978-3-030-50423-6_12.
49. Moustafa N. A new distributed architecture for evaluating AI-based security systems at the edge: network TON_IoT datasets. *Sustain Cities Soc* 2021; 72: 102994. doi: 10.1016/j.scs.2021.102994.
50. Luay M, Layeghy S, Hosseininoorbin S, et al. Temporal analysis of NetFlow datasets for network intrusion detection systems. March 2025.
51. Sharafaldin I, Lashkari AH and Ghorbani AA. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116. SCITEPRESS - Science and Technology Publications. doi: 10.5220/0006639801080116.
52. Ngo T, Yin J, Ge Y-F, et al. Comparative study of machine learning algorithms for IoT cyber threat detection in healthcare information systems. In: Siuly S, Xing C, Li X, et al. (eds) *Health information science*, vol. 15336, Lecture Notes in Computer Science. Springer Nature Singapore, 2025, pp. 68–77. doi: 10.1007/978-981-96-5597-7_7.
53. Lundberg SM and Lee S-I. A unified approach to interpreting model predictions. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 4768–4777. Curran Associates Inc..
54. Akkas S and Azad A. GNNShap: scalable and accurate GNN explanation using shapley values. In: *Proceedings of the ACM Web Conference 2024*, May 2024, pp. 827–838. ACM. doi: 10.1145/3589334.3645599.
55. Sundararajan M, Taly A and Yan Q. Axiomatic attribution for deep networks. 2017, *arXiv*. doi: 10.48550/ARXIV.1703.01365.